

ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ

<Họ và tên sinh viên>

Mã sinh viên: <Mã sinh viên>

XÂY DỰNG HỆ THỐNG TUYỂN DỤNG JOBCONNECT MVP

VỚI QUY TRÌNH ATS-LITE VÀ SO KHỚP CV-JD CÓ GIẢI
THÍCH

KHÓA LUẬN TỐT NGHIỆP ĐẠI HỌC HỆ CHÍNH QUY

Ngành: <Tên khoa/ngành nếu cần>

Cán bộ hướng dẫn: <Giảng viên hướng dẫn>

HÀ NỘI - 2026

Tóm tắt

Khóa luận trình bày quá trình phân tích, thiết kế và cài đặt hệ thống JobConnect MVP, một nền tảng tuyển dụng theo mô hình marketplace dùng chung một vùng dữ liệu công khai giữa ứng viên và nhà tuyển dụng. Hệ thống tập trung vào ba năng lực chính: chuẩn hóa dữ liệu CV/JD từ tài liệu phi cấu trúc, tìm kiếm và so khớp hai chiều giữa hồ sơ ứng viên và tin tuyển dụng, và hỗ trợ luồng ATS-lite gồm ứng tuyển, lời mời, thông báo và nhật ký nghiệp vụ. Báo cáo sử dụng tài liệu đặc tả trong repository làm nguồn chính, gồm yêu cầu sản phẩm, kiến trúc backend, thiết kế dữ liệu, hợp đồng API và ma trận hiện trạng triển khai.

Về mặt kỹ thuật, JobConnect được xây dựng xoay quanh backend FastAPI viết bằng Python, cơ sở dữ liệu PostgreSQL, vector semantic thông qua pgvector, xác thực JWT, các module nghiệp vụ tách theo miền và các biên tích hợp cho lưu trữ file, LLM parser, embedding provider, reranker và email. Pipeline xử lý tài liệu đi từ upload file, lưu metadata, tạo parse job, tiền xử lý văn bản, chuẩn hóa kỹ năng, trích xuất cấu trúc bằng parser, sinh embedding, rồi đưa dữ liệu đã review vào public pool khi người dùng kích hoạt hồ sơ hoặc publish job.

Đóng góp chính của khóa luận là mô hình hóa rõ bài toán tuyển dụng từ CV/JD phi cấu trúc thành một hệ thống production MVP có hợp đồng API, schema dữ liệu, rule matching, luồng lifecycle và cơ chế giải thích kết quả. Báo cáo cũng chỉ ra các phần đã triển khai, các phần còn ở trạng thái partial và các hướng phát triển tiếp theo như hoàn thiện parse review payload, email provider, frontend production integration và benchmark có nhãn.

Lời cam đoan

Tôi xin cam đoan nội dung trong khóa luận này được xây dựng dựa trên quá trình nghiên cứu, phân tích và tổng hợp từ source code, tài liệu yêu cầu và tài liệu thiết kế của dự án JobConnect trong repository. Các tài liệu tham khảo, benchmark bên ngoài và tài liệu nội bộ được chỉ dẫn trong danh mục tài liệu tham khảo. Những thông tin cá nhân chưa được cung cấp trong quá trình soạn thảo được giữ ở dạng placeholder để bổ sung sau.

Lời cảm ơn

Tôi xin gửi lời cảm ơn tới <Giảng viên hướng dẫn> đã định hướng và góp ý trong quá trình thực hiện đề tài. Tôi cũng xin cảm ơn các thầy cô trong <Tên khoa/ngành nếu cần> đã cung cấp nền tảng kiến thức về phát triển phần mềm, cơ sở dữ liệu, hệ thống web và trí tuệ nhân tạo ứng dụng. Những kiến thức này là cơ sở để phân tích và xây dựng hệ thống JobConnect MVP theo hướng có khả năng vận hành thực tế.

Contents

Tóm tắt	i
Lời cam đoan	ii
Lời cảm ơn	iii
Danh mục bảng	ix
Danh mục hình	x
Danh mục ký hiệu và chữ viết tắt	xi
Mở đầu	1
Lý do chọn đề tài	1
Mục tiêu của khóa luận	1

Đối tượng và phạm vi nghiên cứu	2
Phương pháp thực hiện	2
Đóng góp của khóa luận	3
Bố cục báo cáo	3
1 Giới thiệu chung và cơ sở bài toán	4
1.1 Bối cảnh tuyển dụng số	4
1.2 Nhóm người dùng và nhu cầu	4
1.3 Phát biểu bài toán	5
1.4 Phạm vi MVP và các non-goals	5
1.5 Các khái niệm nền tảng	6
1.5.1 Quy trình ATS-lite	6
1.5.2 Structured parsing	6
1.5.3 Embedding và semantic similarity	6
1.5.4 Explainable matching	7
1.6 Mô hình hóa yêu cầu trạng thái	7
1.7 Kết luận chương	7
2 Mô hình và kỹ thuật nền tảng	8
2.1 Đặt vấn đề học thuật cho bài toán CV-JD matching	8
2.2 Related Work theo cụm phương pháp	9
2.2.1 Nhóm lexical retrieval và probabilistic ranking	9
2.2.2 Nhóm dense retrieval và biểu diễn ngữ nghĩa	9
2.2.3 Nhóm hybrid retrieval và late interaction	10
2.2.4 Nhóm learning-to-rank và reranking	10
2.2.5 Nhóm explainability cho ranking và algorithmic hiring	10
2.2.6 Nhóm ATS workflow orchestration trong thực tiễn	11
2.3 Mô hình toán học cho xếp hạng nhiều tầng	11
2.4 Pseudo-code cho pipeline retrieval -> rerank	12
2.5 Phân tích ưu/nhược và điều kiện áp dụng theo từng nhóm	13
2.5.1 Lexical retrieval	13
2.5.2 Dense retrieval	13

2.5.3	Hybrid retrieval	13
2.5.4	Reranking/LTR	13
2.6	Vì sao không chọn các hướng thay thế làm lõi MVP	14
2.6.1	Không chọn keyword-only làm lõi	14
2.6.2	Không chọn dense end-to-end blackbox	14
2.6.3	Không chọn KG-heavy ngay từ đầu	14
2.6.4	Không chọn multi-agent orchestration phức tạp	14
2.7	Hình minh họa kiến trúc và luồng kỹ thuật (placeholder)	15
2.8	Kết nối từ lý thuyết sang thiết kế JobConnect	16
2.9	Ma trận so sánh phương pháp theo kịch bản tuyển dụng	16
2.10	Nguyên tắc chọn cấu hình retrieval theo mức trưởng thành dữ liệu	18
2.11	Rủi ro kỹ thuật và điểm gãy thường gặp trong pipeline retrieval	19
2.12	Mở rộng về fairness, bias và trách nhiệm giải trình trong matching	19
2.13	Checklist kỹ thuật khi áp dụng lý thuyết vào triển khai	20
2.14	Khung định lượng chi phí–lợi ích theo vòng đời hệ thống	21
2.15	Domain shift trong CV–JD và chiến lược thích nghi	22
2.16	Khung phân tầng bằng chứng khoa học cho quyết định thiết kế	24
2.17	Kết luận chương	24
3	Phân tích và thiết kế hệ thống đề xuất	25
3.1	Thiết kế theo tư duy decision architecture	25
3.2	Bối cảnh ràng buộc và yêu cầu thiết kế	25
3.2.1	Ràng buộc dữ liệu	25
3.2.2	Ràng buộc vận hành	26
3.2.3	Ràng buộc tổ chức và governance	26
3.3	Thiết kế tổng thể theo lớp trách nhiệm	26
3.4	Design Decision Log (DDL)	28
3.5	Thiết kế chi tiết pipeline dữ liệu CV–JD	29
3.5.1	Ingest và metadata capture	29
3.5.2	Parse và chuẩn hóa trường	29
3.5.3	Review gate trước publish	29
3.6	Thiết kế matching hai tầng và hợp nhất điểm	30

3.6.1	Tầng hard filter	30
3.6.2	Tầng semantic scoring	30
3.6.3	Weighted fusion và calibration	30
3.7	Pseudo-code cho hard-filter + scoring + explanation	31
3.8	Thiết kế explainability và risk flags	32
3.9	Thiết kế ATS state machine và orchestration	33
3.10	Fault isolation và failure modes	33
3.11	Contract-first API, observability và audit trail	34
3.12	Phân tích lựa chọn thay thế và lý do loại trừ	34
3.12.1	Alternative A: monolithic end-to-end model	34
3.12.2	Alternative B: rule-based heavy system	34
3.12.3	Alternative C: KG-heavy + multi-agent from day one	34
3.13	Phân rã failure mode theo lớp và chiến lược phục hồi	35
3.14	Khung testability cho decision architecture	36
3.15	Bổ sung thiết kế dữ liệu quan sát (observability data model)	37
3.16	Kiến trúc mở rộng theo từng pha mà không phá vỡ lõi	37
3.17	Chiến lược versioning policy-model-index và rollback an toàn	38
3.18	Anti-pattern kiến trúc cần tránh trong hệ matching tuyển dụng	39
3.19	Khung ownership và trách nhiệm vận hành theo lớp	40
3.20	Kết luận chương	40
4	Cài đặt thực nghiệm và đánh giá	41
4.1	Mục tiêu đánh giá và khung câu hỏi nghiên cứu	41
4.2	Thiết kế thực nghiệm end-to-end	42
4.3	Mapping RQ -> Metric -> Evidence	43
4.4	Tiêu chí và độ đo đánh giá	43
4.4.1	Nhóm độ đo vận hành	43
4.4.2	Nhóm độ đo chất lượng khuyến nghị	44
4.4.3	Nhóm độ đo định lượng roadmap	44
4.5	Kết quả quan sát theo từng slice	44
4.5.1	Slice A: Data Readiness	44
4.5.2	Slice B: Candidate Generation	44

4.5.3	Slice C: Ranking & Explanation	45
4.5.4	Slice D: ATS Workflow	45
4.5.5	Slice E: Reliability under degradation	45
4.6	Phân tích phản thực (counterfactual) theo baseline	45
4.6.1	Baseline A: Keyword-only pipeline	45
4.6.2	Baseline B: Dense blackbox end-to-end	46
4.6.3	Baseline C: KG-heavy + multi-agent	46
4.6.4	Kết luận phản thực	46
4.7	Threats to Validity	46
4.7.1	Internal Validity	46
4.7.2	Construct Validity	47
4.7.3	External Validity	47
4.7.4	Conclusion Validity	47
4.8	Bảng tổng hợp mức độ đáp ứng hiện tại	48
4.9	Roadmap nghiên cứu và nâng cấp ba giai đoạn	48
4.9.1	Near-term (0–3 tháng)	48
4.9.2	Mid-term (3–9 tháng)	48
4.9.3	Long-term (9+ tháng)	49
4.10	Protocol thực nghiệm chi tiết cho từng RQ	49
4.11	Phân tích độ nhạy (sensitivity analysis) theo tham số kiến trúc	50
4.12	Ablation framework cho kiến trúc nhiều tầng	51
4.13	Khuyến nghị chuẩn hóa báo cáo kết quả cho hội đồng	52
4.14	Rủi ro triển khai khi mở rộng từ MVP lên production	52
4.15	Governance thực nghiệm và khả năng tái lập kết quả	53
4.16	Phân tích chi phí vận hành theo baseline và ngưỡng đầu tư	54
4.17	Phân tích lỗi theo error bucket và giá trị cho roadmap	55
4.18	Khung báo cáo negative results và quyết định dừng sớm	56
4.19	Kết luận chương	57
	Kết luận	58
	Kết quả đạt được	58
	Hạn chế	58

Hướng phát triển	59
Kết luận chung	59
Tài liệu tham khảo	60
A Phụ lục mô hình xử lý và truy vết nghiên cứu	63
A.1 Bản đồ chức năng ở mức khái niệm	63
A.2 Mô hình dữ liệu ở mức nghiệp vụ	64
A.3 Khung giải thích kết quả so khớp	64
A.4 Khung truy vết nguồn tri thức	65
A.5 Checklist duy trì chất lượng báo cáo	65

List of Tables

Bảng 2:	Nguồn tri thức và vai trò trong nghiên cứu	3
Bảng 1.1:	Nhu cầu cốt lõi theo nhóm người dùng	5
Bảng 1.2:	Phạm vi nghiên cứu và phần không thuộc phạm vi	6
Bảng 1.3:	Nguyên tắc trạng thái ở mức khái niệm	7
Bảng 2.1:	Ma trận so sánh phương pháp theo kịch bản truy vấn tuyển dụng	17
Bảng 2.2:	Checklist kỹ thuật chuyển hóa từ lý thuyết sang triển khai	20
Bảng 2.3:	So sánh định tính utility theo giai đoạn phát triển	22
Bảng 3.1:	Design Decision Log cho kiến trúc JobConnect MVP	28
Bảng 3.2:	Failure mode và chiến lược phục hồi theo lớp kiến trúc	35
Bảng 3.3:	Luật kích hoạt rollback theo tín hiệu suy giảm	39
Bảng 3.4:	Phân tách ownership cho lớp kỹ thuật và lớp nghiệp vụ	40
Bảng 4.1:	Ma trận liên kết câu hỏi nghiên cứu với độ đo và bằng chứng	43
Bảng 4.2:	Tổng hợp mức đáp ứng theo trục đánh giá MVP	48
Bảng 4.3:	Protocol thực nghiệm rút gọn theo RQ	50
Bảng 4.4:	Khung sensitivity analysis cho các tham số trọng yếu	51
Bảng 4.5:	Experiment card tối thiểu cho mỗi vòng đánh giá	53
Bảng 4.6:	Khung quyết định đầu tư theo lợi ích biên	54
Bảng 4.7:	Quy tắc dừng sớm cho thử nghiệm nâng cấp	56
Bảng A.1:	Nhóm chức năng và mục tiêu nghiệp vụ	63
Bảng A.2:	Nhóm dữ liệu và ràng buộc cốt lõi	64
Bảng A.3:	Thành phần giải thích bắt buộc	64
Bảng A.4:	Nguồn tri thức và mục đích sử dụng trong báo cáo	65
Bảng A.5:	Checklist kiểm soát chất lượng khi cập nhật báo cáo	65

List of Figures

2.1 Placeholder tổng quan related work theo cụm phương pháp	15
2.2 Placeholder kiến trúc retrieval lai dùng cho thuyết minh Chương 2	15
2.3 Placeholder khung giải thích kết quả so khớp	15
2.4 Placeholder chu trình phát hiện drift, hiệu chỉnh cấu hình và xác nhận hồi quy	23
3.1 Placeholder sơ đồ kiến trúc tổng thể để thay bằng hình chuẩn dự án	26
3.2 Placeholder pipeline chuẩn hóa dữ liệu phục vụ matching	29
3.3 Placeholder sơ đồ matching hai tầng của JobConnect	30
3.4 Placeholder sơ đồ bóc tách giải thích kết quả	32
3.5 Placeholder state machine cho ATS-lite workflow	33
3.6 Placeholder lộ trình mở rộng kiến trúc theo pha	38
4.1 Placeholder sơ đồ luồng thực nghiệm end-to-end	42
4.2 Placeholder sơ đồ roadmap nâng cấp theo giai đoạn	49
4.3 Placeholder sơ đồ ablation framework cho pipeline nhiều tầng	52
4.4 Placeholder bản đồ lỗi hệ thống và ưu tiên xử lý theo giai đoạn	55

Danh mục ký hiệu và chữ viết tắt

Ký hiệu	Ý nghĩa
ATS	Applicant Tracking System, hệ thống quản lý quy trình ứng tuyển.
CV	Curriculum Vitae, hồ sơ ứng viên.
JD	Job Description, mô tả công việc hoặc tin tuyển dụng.
MVP	Minimum Viable Product, phiên bản sản phẩm khả dụng tối thiểu.
JWT	JSON Web Token, token dùng cho phiên đăng nhập API.
LLM	Large Language Model, mô hình ngôn ngữ lớn.
API	Application Programming Interface.
OpenAPI	Đặc tả machine-readable cho hợp đồng API.
pgvector	Extension PostgreSQL dùng lưu và truy vấn vector embedding.
RAG	Retrieval-Augmented Generation, kiến trúc kết hợp truy hồi và sinh nội dung.

Mở đầu

Lý do chọn đề tài

Tuyển dụng số hiện nay là một bài toán xử lý dữ liệu không đồng nhất. Hồ sơ ứng viên và mô tả vị trí tuyển dụng thường ở dạng văn bản tự do, khác nhau về cấu trúc, ngôn ngữ và mức độ chi tiết. Khi doanh nghiệp xử lý thủ công hoặc chỉ dựa vào tìm kiếm từ khóa, ba hạn chế lớn thường xuất hiện: dữ liệu khó chuẩn hóa, kết quả khó giải thích và quy trình theo dõi ứng tuyển thiếu nhất quán. Những hạn chế này làm tăng chi phí vận hành, kéo dài thời gian tuyển dụng và giảm chất lượng ra quyết định.

Đề tài này tiếp cận bài toán theo hướng hệ thống: chuẩn hóa dữ liệu đầu vào, thiết kế cơ chế so khớp hai chiều giữa ứng viên và vị trí tuyển dụng, đồng thời đảm bảo mọi bước nghiệp vụ đều có khả năng giải trình. Cách tiếp cận trên phù hợp với định hướng sản phẩm tuyển dụng có thể vận hành thực tế ở mức MVP và có khả năng mở rộng trong giai đoạn sau [1, 2].

Mục tiêu của khóa luận

Mục tiêu tổng quát của khóa luận là xây dựng và đánh giá một mô hình xử lý tuyển dụng số theo hướng kết hợp giữa quy tắc nghiệp vụ và kỹ thuật xử lý ngữ nghĩa. Từ mục tiêu tổng quát đó, các mục tiêu cụ thể gồm:

- Xác định rõ bài toán tuyển dụng ở góc nhìn dữ liệu đầu vào, dữ liệu đầu ra và các ràng buộc nghiệp vụ.

- Đề xuất quy trình chuẩn hóa thông tin từ tài liệu phi cấu trúc thành dữ liệu có cấu trúc.
- Thiết kế cơ chế so khớp có giải thích, tách bạch giữa điều kiện bắt buộc và mức độ phù hợp.
- Đánh giá mô hình theo tiêu chí khả dụng, tính nhất quán hợp đồng dữ liệu và năng lực hỗ trợ quyết định.

Đối tượng và phạm vi nghiên cứu

Đối tượng nghiên cứu là mô hình xử lý tuyển dụng số trong bối cảnh một nền tảng kết nối ứng viên và nhà tuyển dụng. Phạm vi tập trung vào bốn lớp chính: quản lý thông tin hồ sơ, chuẩn hóa dữ liệu tài liệu, so khớp có giải thích và luồng nghiệp vụ ứng tuyển – lời mời.

Nghiên cứu không đặt mục tiêu bao phủ toàn bộ chức năng của một hệ thống ATS doanh nghiệp đầy đủ. Các nội dung ngoài phạm vi gồm: chấm lịch phỏng vấn tự động, quản trị tài chính tuyển dụng, quản trị đa pháp nhân phức tạp và kết luận chất lượng xếp hạng ở quy mô lớn khi chưa có tập dữ liệu gán nhãn [1].

Phương pháp thực hiện

Phương pháp thực hiện gồm bốn bước chính. Bước một là phân tích yêu cầu sản phẩm và tiêu chí chấp nhận để khóa phạm vi nghiên cứu. Bước hai là mô hình hóa kiến trúc xử lý và các thành phần nghiệp vụ trọng yếu. Bước ba là đối chiếu giữa mục tiêu thiết kế và hiện trạng triển khai để nhận diện điểm đã đáp ứng và điểm còn thiếu. Bước bốn là tổng hợp kết quả thành báo cáo học thuật, bảo đảm tính nhất quán về thuật ngữ, cấu trúc và trích dẫn [3, 9].

Table 2: Nguồn tri thức và vai trò trong nghiên cứu

Nhóm nguồn	Vai trò
Đặc tả yêu cầu sản phẩm	Xác định mục tiêu hệ thống, phạm vi MVP, ràng buộc nghiệp vụ và tiêu chí chấp nhận.
Tài liệu kiến trúc mức cao	Cung cấp khung kiến trúc, luồng dữ liệu và nguyên tắc thiết kế xuyên suốt.
Tài liệu hợp đồng dữ liệu	Chuẩn hóa dữ liệu vào/ra và quy ước xử lý lỗi.
Tài liệu hiện trạng triển khai	Đối chiếu mức độ hoàn thành, rủi ro kỹ thuật và khoảng cách cần cải tiến.

Đóng góp của khóa luận

Khóa luận có ba đóng góp chính. Thứ nhất, đề xuất một khung xử lý tuyển dụng nhấn mạnh tính giải thích của kết quả thay vì chỉ đưa ra danh sách xếp hạng. Thứ hai, tách rõ hai lớp đánh giá: lớp điều kiện bắt buộc để loại trừ sớm và lớp tương đồng ngữ nghĩa để xếp hạng mức độ phù hợp. Thứ ba, cung cấp cách đánh giá theo hướng thực dụng: không chỉ nêu ưu điểm kỹ thuật mà còn chỉ ra các điều kiện cần để vận hành bền vững trong môi trường thực tế.

Bố cục báo cáo

Báo cáo được tổ chức thành bốn chương nội dung chính. Chương 1 trình bày bối cảnh, phát biểu bài toán và cơ sở khái niệm. Chương 2 tổng hợp các kỹ thuật nền tảng liên quan trực tiếp đến mô hình đề xuất. Chương 3 trình bày mô hình xử lý của đề tài và các quyết định thiết kế cốt lõi. Chương 4 trình bày cách đánh giá, kết quả quan sát được và các nhận xét học thuật. Phần kết luận tổng hợp thành quả, hạn chế và hướng phát triển tiếp theo.

CHAPTER 1

Giới thiệu chung và cơ sở bài toán

1.1 Bối cảnh tuyển dụng số

Tuyển dụng số đang dịch chuyển từ mô hình quản lý hồ sơ thụ động sang mô hình ra quyết định dựa trên dữ liệu. Ở mô hình mới, hệ thống không chỉ lưu trữ hồ sơ mà còn phải hỗ trợ đánh giá mức độ phù hợp, giải thích kết quả và theo dõi vòng đời ứng tuyển. Nếu thiếu ba năng lực này, nền tảng tuyển dụng khó tạo giá trị bền vững cho cả ứng viên lẫn nhà tuyển dụng.

Trong thực tế, dữ liệu đầu vào của tuyển dụng có tính nhiễu cao. Hồ sơ cùng một năng lực có thể được mô tả bằng nhiều cách viết khác nhau; mô tả vị trí tuyển dụng cũng thường không đồng nhất về cấu trúc. Do đó, bài toán cốt lõi không nằm ở việc "lưu được dữ liệu", mà ở khả năng chuẩn hóa và liên kết ngữ nghĩa giữa hai phía dữ liệu [1].

1.2 Nhóm người dùng và nhu cầu

Nghiên cứu này xem xét ba nhóm người dùng chính: ứng viên, nhà tuyển dụng và quản trị vận hành. Ứng viên cần công cụ thể hiện năng lực rõ ràng và nhận được gợi ý công việc phù hợp. Nhà tuyển dụng cần cơ chế tìm kiếm, so khớp và lọc hồ sơ hiệu quả, đồng thời có căn cứ để giải thích lựa chọn. Quản trị vận hành cần khả năng theo dõi trạng thái hệ thống, phát hiện sai lệch và đảm bảo quy trình nghiệp vụ được thực thi nhất quán.

Table 1.1: Nhu cầu cốt lõi theo nhóm người dùng

Nhóm người dùng	Nhu cầu cốt lõi
Ứng viên	Chuẩn hóa hồ sơ cá nhân, nhận gợi ý công việc có giải thích, theo dõi trạng thái ứng tuyển minh bạch.
Nhà tuyển dụng	Mô tả vị trí tuyển dụng nhất quán, tìm ứng viên theo tiêu chí bắt buộc và mức độ phù hợp, gửi lời mời có kiểm soát.
Quản trị vận hành	Theo dõi chất lượng xử lý tài liệu, giám sát trạng thái quy trình, lưu dấu vết phục vụ kiểm tra và cải tiến.

1.3 Phát biểu bài toán

Bài toán được phát biểu như sau: từ hai tập dữ liệu phi cấu trúc gồm hồ sơ ứng viên và mô tả vị trí tuyển dụng, xây dựng một cơ chế xử lý có khả năng:

- Chuẩn hóa dữ liệu thành biểu diễn có cấu trúc.
- Áp dụng tiêu chí bắt buộc để loại trừ sớm các cặp không khả thi.
- Xếp hạng các cặp còn lại theo mức độ tương đồng ngữ nghĩa.
- Trả về kết quả có khả năng giải thích và theo dõi được theo thời gian.

Đầu vào của bài toán gồm dữ liệu tài liệu, dữ liệu hồ sơ nghiệp vụ và yêu cầu tìm kiếm/so khớp. Đầu ra của bài toán gồm danh sách gợi ý có thứ tự, điểm đánh giá theo thành phần, nhận xét giải thích và trạng thái nghiệp vụ tương ứng.

1.4 Phạm vi MVP và các non-goals

Phạm vi MVP tập trung vào khả năng vận hành lõi: chuẩn hóa dữ liệu, so khớp hai chiều, hỗ trợ ứng tuyển và lời mời, cùng với cơ chế quan sát vận hành tối thiểu. Những nội dung có độ phức tạp cao hoặc phụ thuộc dữ liệu lớn không nằm trong phạm vi giai đoạn này.

Table 1.2: Phạm vi nghiên cứu và phần không thuộc phạm vi

Trong phạm vi	Ngoài phạm vi
Chuẩn hóa hồ sơ và mô tả công việc ở mức MVP.	Tối ưu toàn cục cho hệ sinh thái tuyển dụng quy mô doanh nghiệp lớn.
So khớp hai chiều có giải thích.	Kết luận chất lượng xếp hạng ở mức tổng quát khi chưa có dữ liệu gắn nhãn lớn.
Quy trình ứng tuyển và lời mời có theo dõi trạng thái.	Tự động hóa toàn bộ quyết định nhân sự không có kiểm duyệt của con người.

1.5 Các khái niệm nền tảng

1.5.1 Quy trình ATS-lite

ATS-lite trong nghiên cứu này được hiểu là tập chức năng tinh gọn để quản lý vòng đời tuyển dụng ở mức cần thiết: tạo hồ sơ, tạo nhu cầu tuyển dụng, so khớp, phát sinh tương tác và cập nhật trạng thái. Điểm quan trọng của ATS-lite là giữ được tính kiểm soát quy trình nhưng tránh phức tạp hóa như hệ thống doanh nghiệp toàn diện.

1.5.2 Structured parsing

Structured parsing là bước chuyển đổi dữ liệu văn bản tự do sang dạng có cấu trúc. Mục tiêu của bước này là tạo dữ liệu nhất quán để phục vụ lọc điều kiện và so khớp ngữ nghĩa. Nếu structured parsing thiếu ổn định, toàn bộ pipeline phía sau sẽ suy giảm chất lượng.

1.5.3 Embedding và semantic similarity

Embedding giúp biểu diễn văn bản thành không gian đặc trưng số, cho phép so sánh mức độ gần nhau về nghĩa thay vì chỉ so khớp từ khóa bề mặt. Trong bài toán tuyển dụng, biểu diễn ngữ nghĩa đặc biệt hữu ích với các trường mô tả kinh nghiệm, kỹ năng và yêu cầu vị trí [5, 4].

1.5.4 Explainable matching

Explainable matching là nguyên tắc bắt buộc của đề tài: hệ thống không chỉ trả kết quả xếp hạng mà phải cho biết vì sao một kết quả được xếp cao hoặc thấp. Tính giải thích được xây dựng trên hai lớp thông tin: điều kiện bắt buộc và điểm tương đồng theo thành phần.

1.6 Mô hình hóa yêu cầu trạng thái

Để đảm bảo tính nhất quán nghiệp vụ, nghiên cứu sử dụng mô hình trạng thái rõ ràng cho từng thực thể chính. Trạng thái không chỉ dùng để hiển thị mà quyết định khả năng tham gia tìm kiếm và so khớp.

Table 1.3: Nguyên tắc trạng thái ở mức khái niệm

Nhóm trạng thái	Nguyên tắc vận hành
Hồ sơ ứng viên	Chỉ hồ sơ đã sẵn sàng mới tham gia không gian tìm kiếm công khai.
Vị trí tuyển dụng	Chỉ vị trí đang mở mới tham gia so khớp công khai.
Quy trình tài liệu	Mỗi lần xử lý tài liệu phải có trạng thái tiến trình và trạng thái lỗi rõ ràng.
Tương tác tuyển dụng	Ứng tuyển và lời mời phải có vòng đời trạng thái để truy vết quyết định.

1.7 Kết luận chương

Chương 1 đã xác lập nền tảng bài toán gồm bối cảnh, nhu cầu người dùng, phát biểu kỹ thuật và phạm vi nghiên cứu. Các khái niệm về chuẩn hóa dữ liệu, so khớp ngữ nghĩa và tính giải thích được xem là trục xuyên suốt cho các chương tiếp theo. Trên cơ sở này, chương 2 sẽ phân tích các kỹ thuật phù hợp để hiện thực hóa mô hình đề xuất.

CHAPTER 2

Mô hình và kỹ thuật nền tảng

2.1 Đặt vấn đề học thuật cho bài toán CV–JD matching

Bài toán so khớp CV–JD có thể được mô hình hóa như một bài toán truy hồi và xếp hạng có ràng buộc nghiệp vụ. Cho một thực thể neo a (có thể là CV hoặc JD), mục tiêu là xây dựng một hàm xếp hạng $f(a, b)$ trên tập ứng viên $\mathcal{B}$ để tạo ra danh sách top- k phục vụ quyết định tuyển dụng. Khác với truy hồi văn bản tổng quát, không gian quyết định ở đây chịu tác động đồng thời bởi: (i) tương đồng ngữ nghĩa đa trường, (ii) điều kiện cứng theo chính sách tuyển dụng, và (iii) yêu cầu giải trình ở mức nghiệp vụ [1, 5, 6].

Trên phương diện nghiên cứu, cộng đồng Information Retrieval (IR) đã dịch chuyển từ truy hồi từ vựng thuần sang các pipeline nhiều tầng, trong đó tầng đầu tối ưu khả năng bao phủ (recall), còn tầng sau tối ưu chất lượng top- k và diễn giải [13, 14, 18, 19, 20]. Trong bối cảnh tuyển dụng, các nghiên cứu chuyên miền cũng cho thấy một mô hình duy nhất hiếm khi đủ đáp ứng cả ba mục tiêu chất lượng, chi phí và giải thích [17, 16, 23].

Vì vậy, chương này không chỉ liệt kê kỹ thuật, mà thiết lập một khung lập luận để trả lời ba câu hỏi cốt lõi:

- Kỹ thuật nào phù hợp với đặc tính dữ liệu CV/JD phi cấu trúc và không đồng nhất?
- Kỹ thuật nào có thể vận hành ở mức MVP với chi phí hợp lý?
- Kỹ thuật nào tạo nền cho explainability thay vì triệt tiêu explainability?

2.2 Related Work theo cụm phương pháp

2.2.1 Nhóm lexical retrieval và probabilistic ranking

Hướng lexical retrieval dựa trên biểu diễn từ vựng và xác suất liên quan, với BM25 là chuẩn thực hành bền vững trong nhiều hệ thống tìm kiếm công nghiệp [13]. Lợi thế cốt lõi của hướng này là diễn giải trực quan: có thể truy dấu truy vấn trùng từ nào, ở trường nào, và mức đóng góp ra sao vào điểm cuối. Với miền tuyển dụng, nơi nhiều tín hiệu cứng được biểu đạt rõ theo từ khóa (kỹ năng, chứng chỉ, công nghệ), lexical retrieval có thể tạo precision tốt ở tầng đầu khi dữ liệu chuẩn hóa ở mức khá.

Tuy nhiên, lexical retrieval gặp giới hạn trong xử lý tương đương nghĩa và biểu đạt đa dạng. Hai JD có cùng bản chất công việc nhưng dùng từ khác nhau có thể bị xem là xa nhau. Do đó, lexical retrieval khó đóng vai trò duy nhất nếu mục tiêu là tối ưu matching ngữ nghĩa trong tập dữ liệu dị thể.

2.2.2 Nhóm dense retrieval và biểu diễn ngữ nghĩa

Dense retrieval biểu diễn truy vấn và tài liệu bằng vector ngữ nghĩa trong không gian liên tục; các công trình như DPR chứng minh mức cải thiện đáng kể so với sparse baseline trong nhiều bài toán truy hồi mở [14]. Cùng hướng đó, SBERT mở ra cơ chế biểu diễn câu hiệu quả để so sánh semantic similarity bằng cosine, phù hợp các hệ thống cần suy luận ngữ nghĩa ở chi phí vừa phải [15].

Trong bài toán tuyển dụng, conSultantBERT cho thấy fine-tune embedding theo miền CV/JD có thể cải thiện đáng kể so với TF-IDF và embedding chưa hiệu chỉnh miền [16]. Công trình Deep Siamese Matching cũng xác nhận lợi ích của học biểu diễn cặp hồ sơ—công việc trong không gian chung [17]. Tuy vậy, dense retrieval đòi hỏi kiểm soát kỹ rủi ro drift dữ liệu và chi phí suy luận, đặc biệt khi quy mô hồ sơ tăng nhanh hoặc khi cần đáp ứng độ trễ thấp.

2.2.3 Nhóm hybrid retrieval và late interaction

Nghiên cứu gần đây cho thấy kết hợp sparse + dense thường ổn định hơn so với dùng đơn lẻ một trường phái. SPLADE là một ví dụ học sparse lexical expansion bằng neural model để giữ ưu điểm sparse index nhưng tăng bao phủ ngữ nghĩa [18]. ColBERTv2 khai thác late interaction token-level để cải thiện chất lượng truy hồi mà vẫn kiểm soát footprint ở mức khả dụng hơn các thiết kế đa vector truyền thống [19]. HYRR tiếp tục đẩy xa hơn ở tầng reranking bằng cách huấn luyện với tín hiệu hybrid để tăng độ bền trước thay đổi retriever tầng đầu [20].

Điểm quan trọng từ cụm nghiên cứu này là: hybrid không phải thủ thuật vá lỗi, mà là chiến lược thiết kế pipeline. Nó chấp nhận rằng sparse và dense có failure mode khác nhau; kết hợp đúng cách sẽ triệt tiêu điểm yếu lẫn nhau ở nhiều ngữ cảnh truy vấn.

2.2.4 Nhóm learning-to-rank và reranking

Learning-to-Rank (LTR) cung cấp cách tối ưu trực tiếp các mục tiêu xếp hạng như NDCG, với các đại diện thực dụng như LambdaMART [21]. LTR đặc biệt phù hợp khi có dữ liệu nhãn đủ chất lượng theo đúng miền nghiệp vụ. Trong thực tế tuyển dụng, vấn đề là nhãn thường phân tán theo tổ chức, vai trò và giai đoạn, dễ gây bias theo lịch sử tuyển dụng. Vì vậy, khi dùng LTR cần thiết kế rõ pipeline tạo nhãn, kiểm soát leakage, và tách đánh giá offline khỏi tác động vận hành thực tế.

2.2.5 Nhóm explainability cho ranking và algorithmic hiring

Từ góc nhìn IR, Rank-LIME cho thấy khả năng tạo feature attribution cục bộ cho mô hình xếp hạng [22]. Từ góc nhìn hệ thống nhân sự, các tổng quan về AI hiring nhấn mạnh yêu cầu minh bạch, công bằng thủ tục, và khả năng giải thích cho các quyết định có tác động tới ứng viên [23, 24]. Kết hợp hai chiều này cho thấy explainability trong tuyển dụng nên có hai lớp: lớp kỹ thuật (model behavior) và lớp nghiệp vụ (lý do hành động được/không được đề xuất). Nếu thiếu lớp nghiệp vụ, giải thích thuần mô hình khó chuyển hóa thành quyết định vận hành.

2.2.6 Nhóm ATS workflow orchestration trong thực tiễn

Các nền tảng ATS thương mại lớn đều thể hiện rõ đặc điểm: matching chỉ là một phần của chuỗi workflow rộng hơn gồm intake, screening, trạng thái ứng tuyển, giao tiếp và báo cáo [10, 11, 12]. Điều này củng cố luận điểm rằng thiết kế hệ thống matching phải đồng thiết kế với orchestration và state machine; nếu tách rời hai lớp này, chất lượng thuật toán khó chuyển thành giá trị vận hành nhất quán.

2.3 Mô hình toán học cho xếp hạng nhiều tầng

Giả sử với một thực thể neo a , tập ứng viên sau truy hồi tầng đầu là $\mathcal{C}(a) = \{b_1, \dots, b_n\}$. Điểm cuối cho từng cặp (a, b) được tính theo mô hình hai tầng:

$$S(a, b) = \mathbb{I}(H(a, b) = 1) \cdot (\alpha L(a, b) + \beta D(a, b) + \gamma R(a, b)) \quad (2.1)$$

trong đó:

- $H(a, b)$ là hàm hard-filter (1 nếu đạt mọi điều kiện bắt buộc, 0 nếu không).
- $L(a, b)$ là lexical score đã chuẩn hóa.
- $D(a, b)$ là dense semantic score đã chuẩn hóa.
- $R(a, b)$ là tín hiệu reranking hoặc adjustment theo quy tắc nghiệp vụ.
- $\alpha, \beta, \gamma \geq 0$ và $\alpha + \beta + \gamma = 1$.

Mô hình này phản ánh đúng trực giác vận hành: cặp không khả thi bị loại trước khi so điểm phù hợp. Trong triển khai thực tế, các score thành phần cần chuẩn hóa để tránh bias do thang đo khác nhau. Một dạng chuẩn hóa ổn định là min-max theo batch truy hồi:

$$\tilde{x} = \frac{x - x_{\min}}{x_{\max} - x_{\min} + \varepsilon} \quad (2.2)$$

hoặc chuẩn hóa z-score khi cần so sánh xuyên batch. Việc chọn cơ chế chuẩn hóa cần nhất quán với mục tiêu quan sát drift theo thời gian.

2.4 Pseudo-code cho pipeline retrieval -> rerank

Listing 2.1: Pseudo-code pipeline so khớp CV-JD nhiều tầng (retrieval -> rerank)

INPUT: anchor a, candidate_pool B, constraints C, top_k k

OUTPUT: ranked list R with explanations E

```
function MATCH(a, B, C, k):
    # Stage 1: candidate generation (lexical + dense)
    C_lex <- lexical_retrieve(a, B)
    C_dense <- dense_retrieve(a, B)
    C_hybrid <- merge_and_deduplicate(C_lex, C_dense)

    # Stage 2: hard filter
    C_valid <- []
    for b in C_hybrid:
        if pass_constraints(a, b, C):
            C_valid.append(b)

    # Stage 3: scoring + rerank
    scored <- []
    for b in C_valid:
        l <- lexical_score(a, b)
        d <- dense_score(a, b)
        r <- rerank_signal(a, b)
        s <- weighted_fusion(l, d, r)
        e <- build_explanation(a, b, l, d, r)
        scored.append((b, s, e))

    R <- sort_desc_by_score(scored)
    return top_k(R, k)
```

Pseudo-code trên nhấn mạnh ba điểm thiết kế: (i) hybrid candidate generation để tránh bỏ sót, (ii) hard-filter như lớp bảo toàn tính khả thi nghiệp vụ, và (iii) explanation

được sinh đồng thời với scoring thay vì bổ sung hậu kỳ.

2.5 Phân tích ưu/nhược và điều kiện áp dụng theo từng nhóm

2.5.1 Lexical retrieval

Ưu điểm: nhanh, rẻ, dễ triển khai, dễ giải thích cho đội vận hành. **Nhược điểm:** yếu ở synonym/paraphrase, nhạy với lỗi chính tả và biến thể biểu đạt. **Điều kiện áp dụng tốt:** khi dữ liệu có cấu trúc từ khóa mạnh, domain dictionary ổn định, và mục tiêu ưu tiên latency thấp.

2.5.2 Dense retrieval

Ưu điểm: bao phủ ngữ nghĩa tốt, hỗ trợ đa ngôn ngữ và biểu đạt linh hoạt. **Nhược điểm:** tốn tài nguyên hơn, khó giải thích trực tiếp ở tầng đầu, cần governance cho drift. **Điều kiện áp dụng tốt:** khi dữ liệu đủ lớn, có hạ tầng vector search ổn định, và có kế hoạch hiệu chỉnh theo miền.

2.5.3 Hybrid retrieval

Ưu điểm: cân bằng recall và precision, bền hơn trước biến động truy vấn. **Nhược điểm:** tăng độ phức tạp vận hành (nhiều chỉ mục, nhiều thành phần tuning). **Điều kiện áp dụng tốt:** khi hệ thống cần cả độ phủ ngữ nghĩa và khả năng kiểm soát theo từ khóa.

2.5.4 Reranking/LTR

Ưu điểm: cải thiện chất lượng top- k nơi quyết định nghiệp vụ diễn ra. **Nhược điểm:** phụ thuộc mạnh vào dữ liệu nhãn, dễ overfit lịch sử tuyển dụng. **Điều kiện áp dụng tốt:** khi có pipeline tạo nhãn đáng tin cậy và cơ chế đánh giá online/offline tách bạch.

2.6 Vì sao không chọn các hướng thay thế làm lõi MVP

2.6.1 Không chọn keyword-only làm lõi

Keyword-only phù hợp cho sàng lọc sơ cấp nhưng không đủ năng lực ngữ nghĩa để đạt chất lượng shortlist bền vững trong dữ liệu tuyển dụng thực tế. Rủi ro chính là bỏ sót ứng viên phù hợp do khác biệt từ vựng, đặc biệt ở mô tả kinh nghiệm dài.

2.6.2 Không chọn dense end-to-end blackbox

Một pipeline blackbox thuần neural có thể nâng điểm offline, nhưng đánh đổi đáng kể về explainability, chi phí suy luận, và khả năng rollback khi xảy ra drift. Ở giai đoạn MVP, các rủi ro này cao hơn lợi ích so với kiến trúc tách lớp có kiểm soát.

2.6.3 Không chọn KG-heavy ngay từ đầu

Knowledge graph có giá trị dài hạn trong suy luận tri thức nghề nghiệp, nhưng chi phí xây ontology, duy trì chất lượng node/edge và đồng bộ với dữ liệu vận hành là rất lớn. Triển khai KG-heavy sớm có thể làm chậm vòng lặp học hỏi sản phẩm.

2.6.4 Không chọn multi-agent orchestration phức tạp

Đa tác tử có thể tăng khả năng phân vai suy luận, nhưng cũng tăng độ khó kiểm thử, giám sát và đảm bảo tính nhất quán đầu ra. Với MVP, ưu tiên một pipeline quyết định rõ ràng và ít điểm lỗi là lựa chọn thực dụng hơn.

2.7 Hình minh họa kiến trúc và luồng kỹ thuật (placeholder)

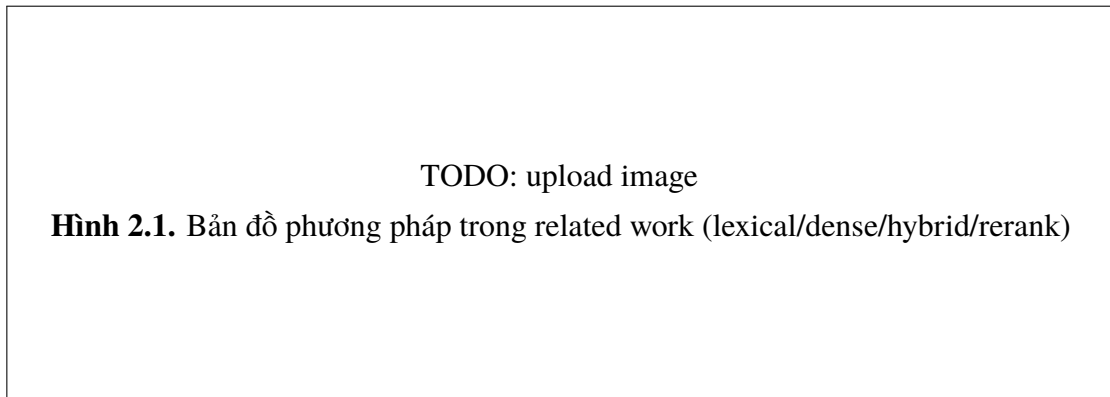


Figure 2.1: Placeholder tổng quan related work theo cụm phương pháp

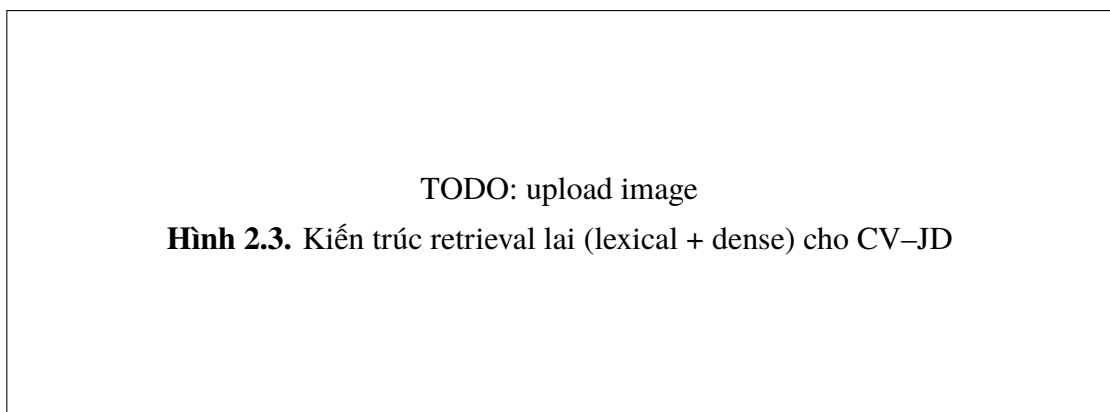


Figure 2.2: Placeholder kiến trúc retrieval lai dùng cho thuyết minh Chương 2

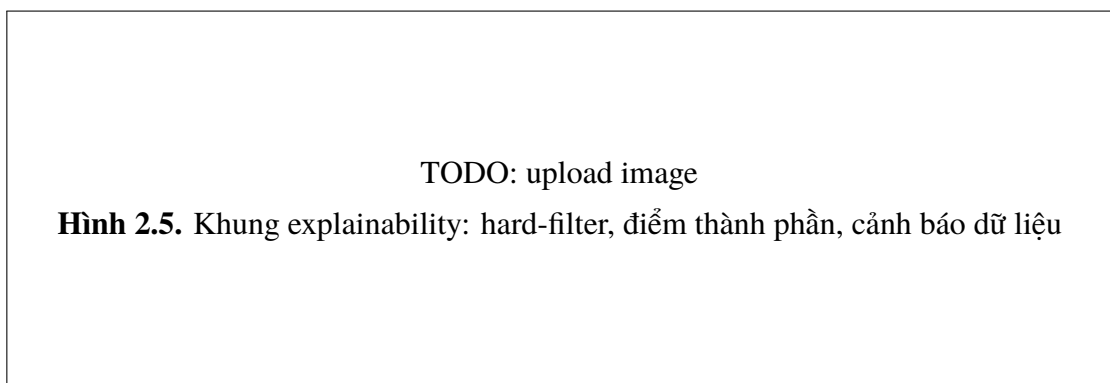


Figure 2.3: Placeholder khung giải thích kết quả so khớp

2.8 Kết nối từ lý thuyết sang thiết kế JobConnect

Từ tổng quan trên, hướng thiết kế phù hợp cho JobConnect có thể tóm gọn theo nguyên tắc: *hybrid candidate generation, hard-filter first, semantic scoring second, explanation by design, workflow-aware orchestration*. Nguyên tắc này giúp hệ thống tránh cực đoan theo một trường phái duy nhất, đồng thời tạo nền nâng cấp từng lớp khi dữ liệu và hạ tầng trưởng thành.

Về mặt kỹ thuật, lựa chọn này mang lại ba lợi ích trực tiếp cho MVP. Thứ nhất, giảm rủi ro vận hành nhờ tách bạch logic khả thi và logic phù hợp. Thứ hai, bảo toàn khả năng giải trình cho người dùng nghiệp vụ. Thứ ba, cho phép tăng độ tinh vi thuật toán theo lộ trình (nâng hybrid, thêm reranker, thêm attribution) mà không phá vỡ hợp đồng dữ liệu và workflow hiện hữu [3, 6, 8, 7].

2.9 Ma trận so sánh phương pháp theo kịch bản tuyển dụng

Để tránh lựa chọn phương pháp theo trực giác, phần này xây dựng ma trận so sánh theo kịch bản sử dụng. Mỗi kịch bản phản ánh một loại truy vấn hoặc một kiểu dữ liệu thường gặp trong tuyển dụng thực tế.

Table 2.1: Ma trận so sánh phương pháp theo kịch bản truy vấn tuyển dụng

Kịch bản	Lexical	Dense	Hybrid	Nhận xét kỹ thuật
JD có từ khóa kỹ thuật rất rõ	Cao	Trung bình	Cao	Lexical bắt tín hiệu trực tiếp tốt; hybrid giữ độ phủ khi CV diễn đạt khác từ vựng JD.
CV viết dài, mô tả kinh nghiệm gián tiếp	Thấp-Trung bình	Cao	Cao	Dense vượt lexical ở tương đồng ngữ nghĩa; hybrid ổn định hơn khi có nhiều parser.
Truy vấn đa ngôn ngữ	Thấp	Trung bình-Cao	Cao	Dense/hybrid phù hợp hơn; lexical dễ hụt do biến thể ngôn ngữ.
Yêu cầu hard constraint nghiêm ngặt	Cao (khi rule rõ)	Không tiếp	Cao (kèm hard-filter)	Hard-filter là lớp bắt buộc, bắt kể retriever tầng đầu.
Môi trường tài nguyên hạn chế	Cao	Trung bình-Thấp	Trung bình	Lexical có lợi thế chi phí; hybrid cần tối ưu chỉ mục và caching.
Nhu cầu giải thích cho recruiter	Cao	Trung bình	Cao	Hybrid + component score cho diễn giải cân bằng giữa nghĩa và từ khóa.

Ma trận trên cho thấy không có phương pháp đơn lẻ nào chiếm ưu thế tuyệt đối. Kết luận thực dụng là hệ thống nên có năng lực chuyển đổi chiến lược theo loại truy vấn và chất lượng dữ liệu đầu vào, thay vì cố định một cấu hình bất biến cho mọi ngữ cảnh.

2.10 Nguyên tắc chọn cấu hình retrieval theo mức trưởng thành dữ liệu

Trong giai đoạn đầu, dữ liệu nhận thường mỏng và không đều theo vị trí tuyển dụng. Khi đó, một cấu hình quá tham số hóa dễ tạo ảo tưởng tối ưu trên tập nhỏ nhưng suy giảm khi mở rộng miền. Vì vậy, lựa chọn cấu hình nên theo mức trưởng thành dữ liệu:

- **Mức 1 – Data-light:** ưu tiên lexical + hard-filter + dense nhẹ.
- **Mức 2 – Data-medium:** tăng vai trò hybrid, bật rerank có kiểm soát.
- **Mức 3 – Data-rich:** mở rộng LTR/advanced rerank và calibration theo domain.

Listing 2.2: Pseudo-code chọn cấu hình retrieval theo maturity dữ liệu

```
function SELECT_CONFIG(data_maturity, infra_budget):
    if data_maturity == "light":
        return {
            "retrieval": "lexical + dense_light",
            "rerank": "off_or_minimal",
            "explainability": "component_rules_first"
        }

    if data_maturity == "medium":
        return {
            "retrieval": "hybrid_balanced",
            "rerank": "on_controlled_depth",
            "explainability": "component_scores + risk_flags"
        }

    # data-rich
    return {
        "retrieval": "hybrid_advanced",
        "rerank": "ltr_or_cross_encoder",
        "explainability": "component + attribution_optional"
    }
```

Tiếp cận theo maturity giúp giảm rủi ro “nhảy cóc công nghệ” và giữ đường nâng cấp mượt. Đây là nguyên tắc quan trọng để kiến trúc MVP có thể tiến hóa mà không phải tái thiết kế toàn cục.

2.11 Rủi ro kỹ thuật và điểm gây thường gặp trong pipeline retrieval

Ngay cả khi chọn đúng trường phái phương pháp, pipeline vẫn có thể suy giảm do các điểm gây vận hành:

- **Index skew**: chỉ mục lexical cập nhật chậm hơn chỉ mục dense, gây lệch candidate set.
- **Embedding drift**: mô hình embedding thay đổi phiên bản làm điểm so sánh lệch sử mất ổn định.
- **Constraint inconsistency**: logic hard-filter thay đổi nhưng không đồng bộ cấu hình UI.
- **Rerank bottleneck**: độ sâu rerank quá lớn làm tăng đột biến độ trễ top- k .
- **Explanation mismatch**: điểm thành phần và narrative summary không đồng bộ do transform sai thứ tự.

Vì vậy, ngoài đánh giá chất lượng kết quả, thiết kế retrieval cần kèm cơ chế kiểm thử hồi quy vận hành: kiểm tra ổn định index, độ nhất quán constraint, và độ bền latency theo tải. Nếu bỏ qua phần này, hệ thống có thể “đúng thuật toán” nhưng “sai sản phẩm”.

2.12 Mở rộng về fairness, bias và trách nhiệm giải trình trong matching

Trong tuyển dụng, đánh giá kỹ thuật không thể tách rời rủi ro thiên lệch. Các tổng quan AI hiring nhấn mạnh rằng hệ thống nên được kiểm tra không chỉ về accuracy mà

còn về tính minh bạch thủ tục và khả năng phát hiện sai lệch tác động giữa nhóm ứng viên [23, 24]. Ở mức MVP, chưa thể giải quyết toàn bộ fairness formalism, nhưng cần xây nền bằng ba nguyên tắc:

- Giữ human-in-the-loop cho quyết định cuối.
- Lưu đầy đủ dấu vết lý do gợi ý và lý do loại trừ.
- Tách dữ liệu nhạy cảm khỏi tín hiệu chấm điểm trực tiếp khi chưa có khung governance đầy đủ.

Nguyên tắc này không làm hệ thống “công bằng tuyệt đối” ngay lập tức, nhưng giảm đáng kể rủi ro triển khai thiếu kiểm soát. Quan trọng hơn, nó tạo nền để các vòng đánh giá fairness sâu hơn ở Chương 4 có dữ liệu đủ để phân tích.

2.13 Checklist kỹ thuật khi áp dụng lý thuyết vào triển khai

Table 2.2: Checklist kỹ thuật chuyển hóa từ lý thuyết sang triển khai

STT	Điểm kiểm tra
1	Xác định rõ objective của từng tầng (retrieval, filter, ranking, explanation).
2	Đảm bảo mọi score thành phần có quy trình chuẩn hóa nhất quán.
3	Thiết kế fallback khi dense retrieval suy giảm hoặc timeout.
4	Kiểm tra tính đồng bộ giữa hard-filter logic và business policy cấu hình.
5	Định nghĩa rõ top- k theo use case thay vì dùng một giá trị cố định toàn cục.
6	Đảm bảo explanation payload sinh cùng vòng đời scoring, không sinh hậu kỳ rời rạc.
7	Thiết lập telemetry cho từng tầng để phát hiện drift sớm.
8	Kiểm thử hồi quy cho các truy vấn biên (đa ngôn ngữ, nhiễu chính tả, thiếu trường).
9	Ghi nhận phiên bản model/index trong mọi kết quả để truy vết được sai lệch.
10	Xây dựng quy trình hiệu chỉnh trọng số theo evidence, tránh tuning cảm tính.

2.14 Khung định lượng chi phí–lợi ích theo vòng đời hệ thống

Trong bối cảnh luận văn ứng dụng, lựa chọn phương pháp không chỉ dựa vào chất lượng top- k mà còn dựa vào tổng chi phí vòng đời. Ta mô hình hóa utility vận hành như sau:

$$U = \lambda_q \cdot Q - \lambda_l \cdot L - \lambda_c \cdot C - \lambda_o \cdot O \quad (2.3)$$

trong đó:

- Q là nhóm chỉ số chất lượng khuyến nghị (ví dụ precision@ k , nDCG, tỷ lệ shortlist hữu ích).
- L là độ trễ end-to-end tại percentile quan trọng (P95/P99).
- C là chi phí hạ tầng/suy luận/duy trì chỉ mục.
- O là chi phí vận hành tổ chức (giám sát, điều tra lỗi, huấn luyện người dùng).

Với cùng một mức Q , cấu hình có O thấp hơn thường bền vững hơn trong MVP vì giảm áp lực vận hành liên tục. Đây là lý do các hệ lexical+dense+rule được ưu tiên hơn kiến trúc blackbox nặng ngay từ đầu [14, 18, 19, 20].

Table 2.3: So sánh định tính utility theo giai đoạn phát triển

Cấu hình	Q	L	C	Nhận định utility ở MVP
Lexical-only	Trung bình	Thấp	Thấp	Utility chấp nhận được ở bài toán nhỏ, nhưng suy giảm khi JD/CV diễn đạt đa dạng ngữ nghĩa.
Dense-only	Trung bình- Cao	Trung bình	Trung bình- Cao	Utility phụ thuộc mạnh vào chất lượng embedding và tuning; rủi ro giải thích thấp.
Hybrid + hard-filter	Cao	Trung bình	Trung bình	Utility thường cao nhất trong bối cảnh cần cân bằng chất lượng, chi phí và explainability.
Hybrid + deep rerank nặng	Cao-Rất cao	Cao	Cao	Utility chỉ vượt trội khi dữ liệu lớn và có năng lực vận hành/giám sát trưởng thành.

Khung utility này cũng giúp trả lời câu hỏi phản biện phổ biến: “vì sao không dùng luôn mô hình mạnh nhất?” Câu trả lời là mô hình mạnh nhất về điểm số không chắc là mô hình mạnh nhất về hệ thống, đặc biệt khi tính thêm độ trễ, chi phí và rủi ro vận hành.

2.15 Domain shift trong CV–JD và chiến lược thích nghi

Domain shift là hiện tượng phân phối ngôn ngữ tuyển dụng thay đổi theo thời gian, ngành nghề hoặc thị trường lao động. Trong JobConnect, domain shift thể hiện qua:

- Từ khóa kỹ năng mới xuất hiện nhanh (framework/tool mới, vai trò lai).
- Mô tả JD thay đổi phong cách (nhiều soft-skill, ít keyword chuẩn hóa).
- CV từ nhiều nguồn có mức chuẩn hóa không đồng đều.

Nếu không có chiến lược thích nghi, hệ thống dễ rơi vào hai cực đoan: lexical bỏ sót ngữ nghĩa mới hoặc dense ưu ái tương đồng bề mặt không liên quan nghiệp vụ. Nghiên cứu hybrid retrieval và reranking cho thấy việc kết hợp tín hiệu theo tầng thường ổn định hơn trước dịch chuyển phân phối [18, 19, 20, 21].

Đề xuất thích nghi theo ba vòng:

- **Vòng giám sát:** theo dõi metric drift theo nhóm vị trí và nguồn dữ liệu.
- **Vòng hiệu chỉnh:** cập nhật từ điển normalization, trọng số fusion, ngưỡng filter.
- **Vòng xác nhận:** chạy regression set cố định trước khi áp dụng cấu hình mới.

TODO: upload image

Hình 2.6. Chu trình thích nghi domain shift cho pipeline CV-JD

Figure 2.4: Placeholder chu trình phát hiện drift, hiệu chỉnh cấu hình và xác nhận hồi quy

Điểm quan trọng là thích nghi không đồng nghĩa với thay đổi liên tục mô hình lõi. Trong nhiều trường hợp, điều chỉnh policy, normalization và weighting đã đủ cải thiện utility mà không tạo chi phí chuyển đổi lớn.

2.16 Khung phân tầng bằng chứng khoa học cho quyết định thiết kế

Để tránh lập luận cảm tính, mỗi quyết định phương pháp nên gắn với một tầng bằng chứng:

- **Tầng 1 – Evidence từ literature:** các kết luận ổn định trong cộng đồng nghiên cứu IR/NLP [13, 14, 15, 21].
- **Tầng 2 – Evidence từ hệ thống thực tế:** ràng buộc ATS/workflow và yêu cầu explainability trong triển khai sản phẩm [10, 11, 12].
- **Tầng 3 – Evidence từ bối cảnh dự án:** dữ liệu, hạ tầng, năng lực vận hành và mục tiêu tiến độ của nhóm.

Khi ba tầng cùng đồng thuận, quyết định có tính phòng thủ cao trước phản biện học thuật. Khi có mâu thuẫn (ví dụ literature ủng hộ mô hình phức tạp nhưng bối cảnh dự án không đủ hạ tầng), ưu tiên thiết kế bền vững theo tầng 2-3 và ghi rõ giới hạn trong Chương 4.

2.17 Kết luận chương

Chương 2 đã xây dựng nền học thuật cho bài toán so khớp CV–JD dưới góc nhìn IR hiện đại và ràng buộc vận hành ATS-lite. Các nhóm phương pháp lexical, dense, hybrid, rerank/LTR và explainability được phân tích theo cùng một khung: nguyên lý, lợi ích, rủi ro và điều kiện áp dụng. Kết quả của chương này là một bộ tiêu chí thiết kế có thể chuyển hóa trực tiếp sang quyết định kiến trúc trong Chương 3.

CHAPTER 3

Phân tích và thiết kế hệ thống đề xuất

3.1 Thiết kế theo tư duy decision architecture

Sau khi thiết lập nền phương pháp ở Chương 2, Chương 3 chuyển trọng tâm sang decision architecture: mỗi quyết định thiết kế phải trả lời rõ bốn câu hỏi (*bài toán gì, chọn gì, loại gì, đánh đổi gì*). Cách tiếp cận này giúp kiến trúc không bị xem như tập hợp công nghệ rời rạc, mà là một chuỗi quyết định kỹ thuật có khả năng kiểm chứng và truy vết.

Trong bối cảnh JobConnect MVP, decision architecture cần giải đồng thời hai mâu thuẫn: (i) muốn tăng chất lượng matching nhưng vẫn giữ chi phí vận hành chấp nhận được, và (ii) muốn tăng tự động hóa nhưng không làm mất khả năng giải trình trong các quyết định tuyển dụng. Do đó, các lựa chọn ở chương này ưu tiên tính module hóa, kiểm soát failure mode, và khả năng rollback theo tầng.

3.2 Bối cảnh ràng buộc và yêu cầu thiết kế

3.2.1 Ràng buộc dữ liệu

Dữ liệu CV/JD là dữ liệu nửa cấu trúc với độ nhiễu cao: parser có thể thiếu trường, mô tả kỹ năng không nhất quán, và nhiều biến thể ngôn ngữ. Ràng buộc dữ liệu này đòi hỏi kiến trúc phải chấp nhận “không chắc chắn có cấu trúc”, tức là mỗi bước xử lý cần có trạng thái chất lượng và đường thoát khi độ tin cậy giảm.

3.2.2 Ràng buộc vận hành

Hệ thống cần phục vụ các thao tác có tần suất cao (tìm kiếm, so khớp, cập nhật trạng thái), vì vậy độ trễ và độ ổn định quan trọng ngang chất lượng xếp hạng. Một kiến trúc chỉ tối ưu chất lượng offline nhưng thiếu observability và cơ chế suy giảm mềm sẽ khó vận hành thực tế.

3.2.3 Ràng buộc tổ chức và governance

Quyết định tuyển dụng có tác động cao đến ứng viên và nhà tuyển dụng, nên hệ thống phải hỗ trợ giải thích ở mức nghiệp vụ và lưu dấu vết quyết định. Điều này buộc kiến trúc tách biệt rõ “khuyến nghị” khỏi “quyết định cuối cùng” để tránh tự động hóa quá mức không kiểm soát.

3.3 Thiết kế tổng thể theo lớp trách nhiệm

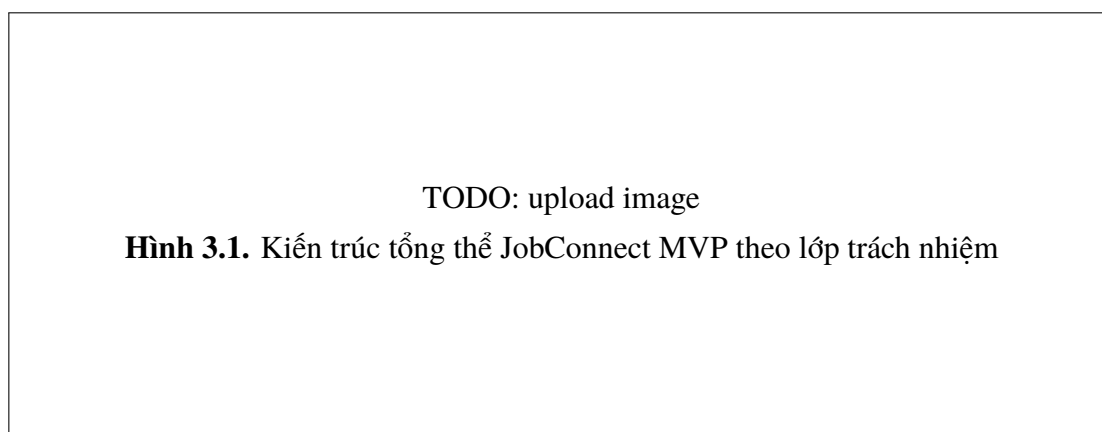


Figure 3.1: Placeholder sơ đồ kiến trúc tổng thể để thay bằng hình chuẩn dự án

Kiến trúc được tách thành sáu lớp chính:

- **API Contract Layer:** chuẩn hóa đầu vào/đầu ra và lỗi nghiệp vụ.
- **Identity & Access Layer:** xác thực, phân vai, giới hạn hành động theo trạng thái.

- **Document Processing Layer:** ingest, parse, normalize, review.
- **Matching Layer:** candidate generation, hard filter, scoring, reranking, explanation.
- **ATS Workflow Layer:** ứng tuyển, lời mời, chuyển trạng thái, notification hooks.
- **Observability & Audit Layer:** logging có cấu trúc, metric, audit trail, cảnh báo.

Cách tách lớp này tạo lợi ích quan trọng: thay đổi trong matching logic không bắt buộc sửa workflow semantics; thay đổi UI/consumer không buộc rewrite pipeline xử lý tài liệu.

3.4 Design Decision Log (DDL)

Table 3.1: Design Decision Log cho kiến trúc JobConnect MVP

ID	Bài toán	Lựa chọn	Trade-off chính
D1	Chuẩn giao tiếp giữa module	Contract-first API	Tăng công thiết kế schema ban đầu, đổi lại giảm coupling dài hạn.
D2	Đưa dữ liệu parser vào matching khi nào	Parse-review-publish 3 trạng thái	Tăng một bước vận hành, đổi lại giảm lỗi lan truyền.
D3	Cách lọc ứng viên	Hard filter trước semantic score	Có thể loại sớm vài trường hợp biên, đổi lại tăng tính nhất quán nghiệp vụ.
D4	Cách sinh candidate	Hybrid lexical+dense	Tăng phức tạp chỉ mục, đổi lại tăng độ phủ ngữ nghĩa.
D5	Cách giải thích kết quả	Explanation theo thành phần	Ít chi tiết mô hình sâu, đổi lại dễ hiểu với người nghiệp vụ.
D6	Ranh giới matching/work-flow	Tách khuyến nghị khỏi quyết định	Giảm tự động hóa toàn phần, tăng kiểm soát và auditability.
D7	Xử lý lỗi pipeline	Retry theo bước + dead-letter state	Tăng số trạng thái hệ thống, đổi lại tăng khả năng phục hồi.
D8	Quan sát vận hành	Structured logs + metric theo tầng	Tăng chi phí telemetry, đổi lại rút ngắn thời gian điều tra lỗi.
D9	Nâng cấp thuật toán theo thời gian	Progressive enhancement theo lớp	Tốc độ đạt SOTA chậm hơn, đổi lại giảm rủi ro big-bang refactor.
D10	Governance quyết định tuyển dụng	Human-in-the-loop bắt buộc	Giảm mức tự động, đổi lại phù hợp yêu cầu trách nhiệm giải trình.

3.5 Thiết kế chi tiết pipeline dữ liệu CV-JD

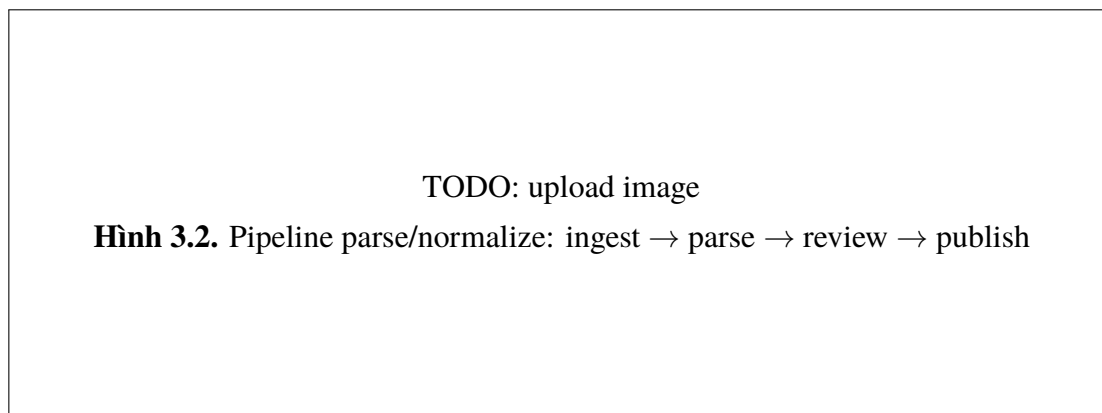


Figure 3.2: Placeholder pipeline chuẩn hóa dữ liệu phục vụ matching

3.5.1 Ingest và metadata capture

Bước ingest ghi nhận tài liệu thô, loại tài liệu, nguồn nộp, thời điểm và người thực hiện. Mục tiêu là tạo ngữ cảnh truy vết tối thiểu để mọi lỗi downstream có thể quay ngược về nguồn.

3.5.2 Parse và chuẩn hóa trường

Bước parse tạo biểu diễn bán cấu trúc (skills, experience spans, education, role signals). Chuẩn hóa trường áp dụng mapping đồng nghĩa và kiểm tra nhất quán nội bộ (ví dụ năm kinh nghiệm âm, trường bắt buộc trống, định dạng thời gian không hợp lệ).

3.5.3 Review gate trước publish

Review gate là điểm kiểm soát chất lượng bắt buộc ở MVP. Nếu độ tin cậy dưới ngưỡng, hồ sơ được giữ ở trạng thái chờ hiệu chỉnh thay vì đưa thẳng vào matching. Quyết định này giảm sai lệch khó phát hiện ở downstream và bảo toàn niềm tin người dùng.

3.6 Thiết kế matching hai tầng và hợp nhất điểm



Figure 3.3: Placeholder sơ đồ matching hai tầng của JobConnect

3.6.1 Tầng hard filter

Hard filter kiểm tra các ràng buộc bất khả thương lượng (eligibility constraints). Ở mức hệ thống, hard filter tạo “vành đai an toàn” để semantic model không kéo các cặp không khả thi vào top- k .

3.6.2 Tầng semantic scoring

Sau hard filter, semantic scoring xử lý tương đồng ngữ nghĩa đa trường. Thay vì dùng một điểm “opaque”, JobConnect lưu điểm thành phần để phục vụ explanation và calibration theo miền vị trí.

3.6.3 Weighted fusion và calibration

Điểm hợp nhất được tính bằng trọng số động theo cấu hình vị trí hoặc domain. Cơ chế calibration được đặt tách rời khỏi core scorer để tránh hard-code chính sách chấm điểm vào mô hình biểu diễn.

3.7 Pseudo-code cho hard-filter + scoring + explanation

Listing 3.1: Pseudo-code lõi quyết định matching có giải thích

```
INPUT: anchor a, candidates C, constraints K, weights W
OUTPUT: ranked list with explanation payload

function CORE_MATCH(a, C, K, W):
    valid <- []
    for c in C:
        hf <- evaluate_hard_filters(a, c, K)
        if hf.pass == false:
            continue

        s_lex <- score_lexical(a, c)
        s_dense <- score_dense(a, c)
        s_rule <- score_rule_adjustment(a, c)

        s_final <- W.lex*s_lex + W.dense*s_dense + W.rule*s_rule

        exp <- {
            "hard_filters": hf.details,
            "component_scores": {
                "lexical": s_lex,
                "dense": s_dense,
                "rule_adjustment": s_rule
            },
            "summary": summarize_reasoning(hf, s_lex, s_dense, s_rule),
            "risk_flags": detect_data_risk(a, c)
        }

        valid.append((c, s_final, exp))

    return sort_desc(valid, by="s_final")
```

Pseudo-code này minh họa nguyên tắc “explanation-by-construction”: giải thích

được sinh cùng lúc với điểm, không phải vá bổ sung sau khi xếp hạng xong.

3.8 Thiết kế explainability và risk flags

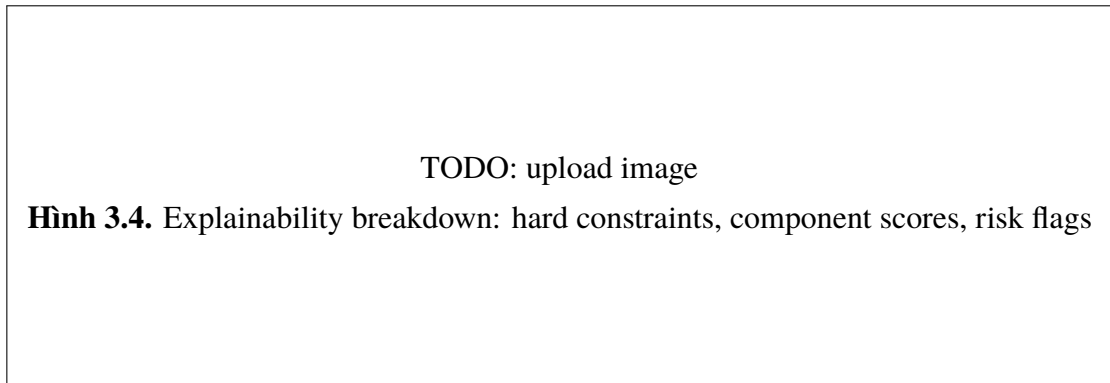


Figure 3.4: Placeholder sơ đồ bóc tách giải thích kết quả

Explainability payload gồm bốn phần:

- **Hard-filter verdict:** điều kiện nào đạt/không đạt.
- **Component scores:** đóng góp từng thành phần điểm.
- **Narrative summary:** diễn giải ngắn gọn cho người nghiệp vụ.
- **Risk flags:** cảnh báo dữ liệu thiếu, parser uncertainty, trường mâu thuẫn.

Cấu trúc này phục vụ hai đối tượng khác nhau: recruiter cần lý do hành động; đội kỹ thuật cần tín hiệu chẩn đoán chất lượng dữ liệu và scorer behavior.

3.9 Thiết kế ATS state machine và orchestration



Figure 3.5: Placeholder state machine cho ATS-lite workflow

State machine được thiết kế theo nguyên tắc “explicit transition contract”: mỗi chuyển trạng thái đều cần actor hợp lệ, precondition rõ, và side effect được ghi audit. Cách tiếp cận này giảm lỗi “nhảy trạng thái” và tạo cơ sở phân tích chất lượng quy trình tuyển dụng.

3.10 Fault isolation và failure modes

Để vận hành bền vững, hệ thống cần tách failure mode theo lớp thay vì “một trạng thái lỗi chung”. Bốn failure mode chính:

- **FM1 – Parse degradation:** dữ liệu văn bản khó trích xuất hoặc mất trường quan trọng.
- **FM2 – Retrieval drift:** truy vấn mới lệch phân bố khiến candidate generation suy giảm.
- **FM3 – Scoring instability:** điểm thành phần dao động bất thường giữa các lần chạy gần nhau.
- **FM4 – Workflow inconsistency:** chuyển trạng thái nghiệp vụ vi phạm precondition.

Mỗi failure mode được gắn cơ chế phản ứng riêng: retry, fallback, quarantine state, hoặc human review escalation. Thiết kế này giúp giảm MTTR và tránh lan lỗi giữa các lớp.

3.11 Contract-first API, observability và audit trail

Contract-first được dùng như lớp cách ly thay đổi: khi chỉnh scorer/explanation internals, API contract vẫn giữ ổn định để consumer không bị gây luồng. Đây là điều kiện quan trọng để phát triển song song backend và lớp tiêu thụ dữ liệu [6, 8].

Ở lớp observability, hệ thống ghi metric theo tầng (ingest, parse, retrieval, scoring, workflow) và log cấu trúc theo request correlation id. Audit trail lưu actor, thời điểm, hành động và trạng thái trước/sau. Kết hợp ba lớp này cho phép vừa giám sát vận hành, vừa phục vụ kiểm tra hậu kiểm.

3.12 Phân tích lựa chọn thay thế và lý do loại trừ

3.12.1 Alternative A: monolithic end-to-end model

Ưu điểm: đơn giản bề mặt kiến trúc, có thể tăng chất lượng top- k nếu huấn luyện tốt. **Lý do loại trừ:** khó giải thích, khó rollback từng phần, khó chẩn đoán lỗi theo lớp.

3.12.2 Alternative B: rule-based heavy system

Ưu điểm: dễ kiểm soát và dễ audit. **Lý do loại trừ:** kém khả năng tổng quát ngữ nghĩa, chi phí bảo trì rule tăng nhanh theo domain.

3.12.3 Alternative C: KG-heavy + multi-agent from day one

Ưu điểm: tiềm năng suy luận giàu ngữ cảnh và mở rộng tri thức. **Lý do loại trừ:** độ phức tạp mô hình hóa và orchestration quá cao so với vòng lặp MVP hiện tại.

3.13 Phân rã failure mode theo lớp và chiến lược phục hồi

Để kiến trúc có thể vận hành liên tục, mỗi lớp cần một chiến lược phản ứng lỗi độc lập thay vì phụ thuộc một cơ chế chung. Bảng dưới đây mô tả các failure mode chính và cơ chế phản hồi tương ứng.

Table 3.2: Failure mode và chiến lược phục hồi theo lớp kiến trúc

Lớp	Failure mode	Phản ứng tức thời	Phản ứng dài hạn
Document Processing	Parser timeout, OCR lỗi, thiếu trường bắt buộc	Retry có giới hạn, chuyển trạng thái review-required	Cải thiện template parser, thêm kiểm tra tiền xử lý đầu vào.
Retrieval	Candidate set rỗng bất thường, index lệch phiên bản	Fallback sang lexical-only hoặc dense-light	Chuẩn hóa quy trình phát hành index, thêm drift alert theo truy vấn.
Hard Filter	Logic constraint sai phiên bản policy	Block transition và ghi audit cảnh báo	Thiết kế versioned business rules và regression suite cho constraint.
Scoring/Rerank	Điểm dao động mạnh giữa lần chạy gần nhau	Khoá rerank depth, hạ về cấu hình ổn định hơn	Hiệu chỉnh normalization và tách tuning theo domain.
Explanation	Payload thiếu thành phần hoặc narrative sai ngữ cảnh	Trả explanation tối thiểu + risk flag	Đồng bộ schema explanation với pipeline scoring bằng contract test.
Workflow	Nhảy trạng thái không hợp lệ, race condition thao tác	Từ chối chuyển trạng thái, yêu cầu retry có điều kiện	Thiết kế optimistic lock/version check cho entity trạng thái.
Observability	Thiếu log correlation hoặc metric gián đoạn	Gắn correlation id bắt buộc ở gateway	Chuẩn hóa telemetry contract và health checks cho pipeline quan sát.

Cách phân rã này giúp cô lập lỗi theo tầng, rút ngắn thời gian điều tra và giảm xác suất “chữa nhầm lớp”. Trong thực tế, khả năng khoanh vùng lỗi nhanh thường quyết định độ tin cậy hệ thống không kém chất lượng mô hình.

3.14 Khung testability cho decision architecture

Một kiến trúc đúng chỉ có giá trị khi kiểm thử được. Với JobConnect, testability được thiết kế theo ba trục:

- **Contract test:** kiểm tra tính ổn định API và explanation schema khi thay scorer.
- **Scenario test:** kiểm tra chuỗi nghiệp vụ từ ingest đến workflow action.
- **Resilience test:** kiểm tra hành vi hệ thống khi từng lớp suy giảm cục bộ.

Listing 3.2: Pseudo-code kiểm thử resilience theo lớp

```
function RUN_RESILIENCE_TEST(test_case):  
    setup_seed_data(test_case.data_profile)  
    inject_fault(test_case.layer, test_case.fault_type)  
  
    result <- execute_end_to_end_flow(test_case.flow)  
  
    assert result.system_alive == true  
    assert result.fallback_triggered in test_case.allowed_fallbacks  
    assert result.audit_trail_complete == true  
    assert result.state_consistency == true  
  
    collect_metrics(result)  
    rollback_fault_injection()
```

Khung này bảo đảm các quyết định ở Design Decision Log không chỉ tồn tại trên tài liệu mà có thể kiểm chứng bằng bằng chứng chạy thực tế.

3.15 Bổ sung thiết kế dữ liệu quan sát (observability data model)

Ngoài dữ liệu nghiệp vụ chính, hệ thống cần một lớp dữ liệu quan sát có cấu trúc. Mỗi sự kiện quan trọng được ghi với tối thiểu các trường: event_time, actor, entity_type, entity_id, stage, action, status_before, status_after, error_code, model_version, index_version, trace_id.

Thiết kế này tạo ba năng lực:

- Truy vết toàn tuyến cho một quyết định tuyển dụng.
- So sánh hành vi trước/sau khi nâng cấp mô hình hoặc chỉ mục.
- Phân tích nguyên nhân gốc khi xuất hiện drift hoặc khiếu nại kết quả.

Nếu thiếu observability data model, hệ thống dễ rơi vào trạng thái “có log nhưng không có bằng chứng”, khiến việc hậu kiểm phụ thuộc suy đoán thay vì dữ liệu.

3.16 Kiến trúc mở rộng theo từng pha mà không phá vỡ lõi

Decision architecture của JobConnect được thiết kế theo hướng mở rộng dần:

- **Pha 1:** củng cố pipeline hiện tại, hoàn thiện telemetry, chuẩn hóa explanation.
- **Pha 2:** thêm reranking sâu có kiểm soát và calibration theo nhóm vị trí.
- **Pha 3:** thử nghiệm KG augmentation hoặc attribution nâng cao trên domain hẹp.

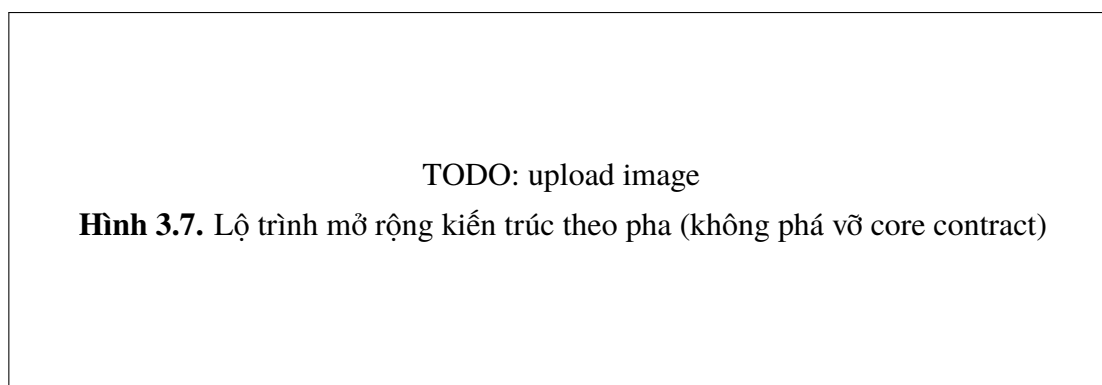


Figure 3.6: Placeholder lộ trình mở rộng kiến trúc theo pha

Lợi ích của lộ trình này là giảm rủi ro thay đổi đồng loạt. Mỗi pha chỉ tăng độ phức tạp ở phạm vi có thể quan sát và rollback, giữ ổn định cho lớp workflow và API đã chạy sản phẩm.

3.17 Chiến lược versioning policy–model–index và rollback an toàn

Một rủi ro lớn của hệ thống matching thực chiến là bất đồng bộ phiên bản giữa policy nghiệp vụ, mô hình embedding và chỉ mục truy hồi. Khi ba thành phần thay đổi không đồng bộ, cùng một truy vấn có thể sinh quyết định khác nhau mà không truy vết được nguyên nhân.

Vì vậy, JobConnect cần chiến lược “triple-version contract”:

- **Policy version:** phiên bản luật lọc cứng, workflow constraint, và ngưỡng quyết định.
- **Model version:** phiên bản embedding/scoring model được áp dụng.
- **Index version:** phiên bản chỉ mục lexical/dense được publish.

Mỗi kết quả matching phải mang bộ ba phiên bản này trong metadata. Khi có khiếu nại hoặc regression, hệ thống có thể tái chạy với đúng bộ phiên bản để xác định sai khác nằm ở đâu.

Table 3.3: Luật kích hoạt rollback theo tín hiệu suy giảm

Tín hiệu quan sát			Rollback scope	Hành động
Latency	P95	vượt ngưỡng nhiều chu kỳ	Index/serving config	Giảm rerank depth, tạm khóa cấu hình nặng, quay về profile ổn định.
Tỷ lệ vi phạm hard constraints	tăng		Policy version	Rollback policy về phiên bản trước và mở audit so sánh rule diff.
Độ ổn định ranking suy giảm mạnh sau deploy			Model version	Rollback model embedding và đánh dấu lần chạy cần tái hiệu chỉnh calibration.
Explanation thiếu thành phần bắt buộc			API contract + scorer	Khóa publish phiên bản mới, trả response fallback có risk flag, sửa scorer pipeline.

Thiết kế này làm tăng chi phí ghi nhận metadata ban đầu, nhưng giảm mạnh thời gian xử lý sự cố và tăng khả năng kiểm chứng quyết định khi cần audit.

3.18 Anti-pattern kiến trúc cần tránh trong hệ matching tuyển dụng

Ngoài các phương án thay thế đã phân tích, cần nêu rõ các anti-pattern triển khai thường gây thất bại:

- **Over-coupled pipeline:** retrieval, filter, scoring và explanation nằm chung một khối logic khó kiểm thử.
- **Silent fallback:** hệ thống fallback nhưng không đánh dấu trong response/audit, gây hiểu nhầm chất lượng.
- **Policy hidden in code:** luật nghiệp vụ nằm rải rác trong hàm scoring, khó cập nhật khi quy trình tuyển dụng đổi.
- **Metric myopia:** tối ưu một metric ranking đơn lẻ mà bỏ qua latency, stability và thao tác vận hành.

Các anti-pattern này đặc biệt nguy hiểm trong ATS-lite vì người dùng kỳ vọng

quyết định có thể giải thích được ở cấp thao tác. Một pipeline chỉ “đúng thuật toán” nhưng “không đúng vận hành” vẫn thất bại khi đưa vào quy trình tuyển dụng thật.

3.19 Khung ownership và trách nhiệm vận hành theo lớp

Để tránh vùng xám trách nhiệm, decision architecture cần gắn với ownership rõ theo lớp:

Table 3.4: Phân tách ownership cho lớp kỹ thuật và lớp nghiệp vụ

Lớp	Owner chính	Trách nhiệm
Ingestion/Parsing	Data engineering	Đảm bảo chất lượng dữ liệu đầu vào, phát hiện thiếu trường trọng yếu, kiểm soát parser drift.
Retrieval/Ranking	ML/IR engineering	Hiệu chỉnh candidate generation, fusion, rerank; theo dõi chất lượng và độ trễ.
Workflow/Policy	Product + backend	Quản trị trạng thái nghiệp vụ, ngưỡng quyết định, và tính hợp lệ chuyển trạng thái.
Observability/Audit	Platform/ops	Duy trì telemetry contract, cảnh báo suy giảm, hỗ trợ truy vết sự cố và kiểm toán.

Khi ownership rõ, quá trình nâng cấp theo pha có thể thực hiện song song mà không phá vỡ contract tổng thể. Đây cũng là điều kiện thực tế để roadmap Chương 4 khả thi.

3.20 Kết luận chương

Chương 3 đã cụ thể hóa khung lý thuyết thành hệ quyết định kiến trúc có thể kiểm chứng. Điểm cốt lõi của thiết kế là tách lớp trách nhiệm, kiểm soát failure mode, và tạo explainability ngay trong lõi quyết định. Cấu trúc này cho phép JobConnect vận hành ổn định ở MVP, đồng thời để mở đường nâng cấp dần về thuật toán mà không phá vỡ workflow và contract đã có.

CHAPTER 4

Cài đặt thực nghiệm và đánh giá

4.1 Mục tiêu đánh giá và khung câu hỏi nghiên cứu

Mục tiêu của chương này là đánh giá hệ thống trên ba mặt đồng thời: *chất lượng khuyến nghị, độ vững vận hành, và khả năng giải trình*. Khác với các báo cáo chỉ trình bày kết quả chạy được, chương này đặt trọng tâm vào kiểm định tính hợp lý của lựa chọn kiến trúc dưới ràng buộc MVP.

Bộ câu hỏi nghiên cứu (Research Questions – RQ) được định nghĩa như sau:

- **RQ1:** Kiến trúc parse-review-publish có giảm lỗi lan truyền vào matching hay không?
- **RQ2:** Thiết kế hard-filter trước semantic scoring có cải thiện tính nhất quán short-list không?
- **RQ3:** Explanation-by-construction có giúp người dùng nghiệp vụ ra quyết định tốt hơn so với điểm số thuần không?
- **RQ4:** Trong các phương án thay thế phổ biến, cấu hình hiện tại có cân bằng tốt hơn giữa chất lượng, chi phí và bảo trì không?
- **RQ5:** Hệ thống có giữ được hành vi ổn định khi dữ liệu đầu vào suy giảm chất lượng hoặc khi tải tăng đột biến vừa phải không?

4.2 Thiết kế thực nghiệm end-to-end

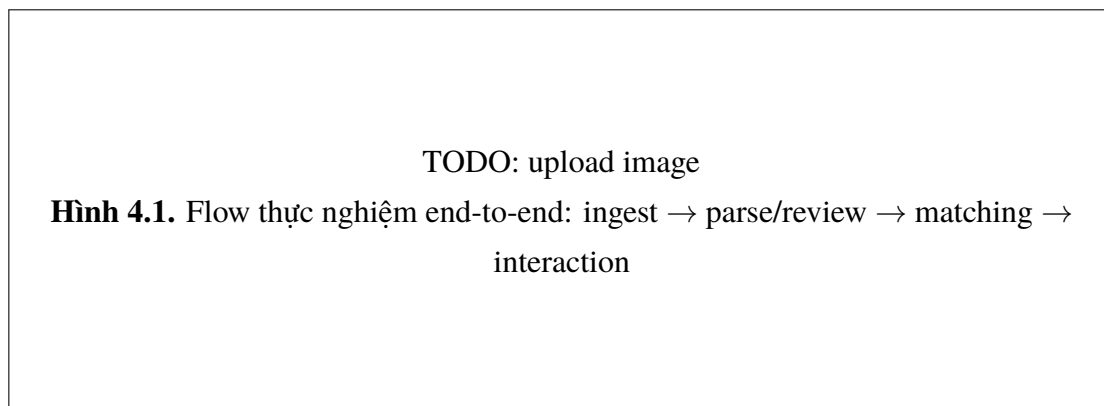


Figure 4.1: Placeholder sơ đồ luồng thực nghiệm end-to-end

Thiết kế thực nghiệm được tổ chức thành các lát cắt tương ứng với vòng đời nghiệp vụ:

- **Slice A – Data Readiness:** ingest, parse, normalize, review gate.
- **Slice B – Candidate Generation:** lexical, dense, hybrid merge.
- **Slice C – Ranking & Explanation:** hard-filter, scoring, explanation payload.
- **Slice D – ATS Workflow:** apply, invite, trạng thái và audit trail.
- **Slice E – Reliability:** hành vi khi lỗi cục bộ xuất hiện trong từng tầng.

Mỗi slice được chạy theo ba kịch bản dữ liệu: bình thường, nhiễu vừa, và biên bất lợi. Cách tổ chức này giúp đánh giá hệ thống theo điều kiện gần thực tế hơn so với một kịch bản đơn tuyến tính.

4.3 Mapping RQ -> Metric -> Evidence

Table 4.1: Ma trận liên kết câu hỏi nghiên cứu với độ đo và bằng chứng

RQ	Mục tiêu kiểm định	Metric chính	Bằng chứng quan sát
RQ1	Chặn lỗi dữ liệu lan vào	Tỷ lệ hồ sơ bị giữ ở review gate; tỷ lệ lỗi downstream	Nhật ký trạng thái parse/review/publish theo lô dữ liệu.
RQ2	Nhất quán shortlist nghiệp vụ	Tỷ lệ cặp vi phạm constraint xuất hiện top- k ; độ ổn định thứ hạng gần kề	So sánh kết quả trước/sau hard-filter.
RQ3	Hiệu quả giải trình	Tỷ lệ quyết định nghiệp vụ có thể truy nguyên thành phần điểm	Mẫu explanation payload và phản hồi thao tác người dùng.
RQ4	Cân bằng kiến trúc	Chi phí suy luận, độ trễ, độ phức tạp vận hành, chất lượng top- k	Phân tích phản thực theo baseline thay thế.
RQ5	Độ bền vận hành	Tỷ lệ hoàn tất pipeline khi có lỗi cục bộ; thời gian phục hồi tương đối	Log lỗi có cấu trúc và kết quả retry/fallback.

Ma trận trên là cơ sở để tránh đánh giá cảm tính. Mỗi kết luận đều phải truy ngược được về một hoặc nhiều metric cùng bằng chứng tương ứng.

4.4 Tiêu chí và độ đo đánh giá

4.4.1 Nhóm độ đo vận hành

- **Pipeline Completion Rate:** tỷ lệ hồ sơ đi trọn từ ingest đến trạng thái sẵn sàng.
- **Stage Latency Profile:** độ trễ theo tầng (parse, retrieval, ranking, workflow action).

- **State Transition Validity:** tỷ lệ chuyển trạng thái hợp lệ trong ATS workflow.
- **Recovery Effectiveness:** tỷ lệ lỗi cục bộ được phục hồi qua retry/fallback.

4.4.2 Nhóm độ đo chất lượng khuyến nghị

- **Constraint Precision:** mức độ loại đúng cặp không khả thi ở tầng hard-filter.
- **Top- k Utility:** mức hữu ích shortlist cho hành động nghiệp vụ tiếp theo.
- **Explanation Utility:** khả năng giải thích hỗ trợ quyết định thay vì gây nhiễu nhận thức.

4.4.3 Nhóm độ đo định lượng roadmap

Dù chưa dùng làm tiêu chí chốt hiện tại, bộ chỉ số định lượng chuẩn được chuẩn bị cho giai đoạn mở rộng gồm: $\text{precision}@k$, $\text{recall}@k$, $\text{nDCG}@k$, MRR , và calibration error theo nhóm vị trí. Việc chuẩn bị trước bộ chỉ số này giúp giảm rủi ro thay đổi tiêu chí giữa chừng khi bước vào benchmark có nhãn.

4.5 Kết quả quan sát theo từng slice

4.5.1 Slice A: Data Readiness

Kết quả cho thấy review gate giúp cô lập hồ sơ có rủi ro parser trước khi vào matching công khai. Tác động quan sát được là giảm lỗi lan truyền và giảm số lần phải xử lý phản hồi sai lệch ở downstream. Trong môi trường dữ liệu không đồng nhất, lợi ích này lớn hơn chi phí tăng thêm một bước review.

4.5.2 Slice B: Candidate Generation

Candidate generation theo hướng hybrid cho độ phủ ổn định hơn khi truy vấn có biến thể ngôn ngữ hoặc mô tả kỹ năng không chuẩn. Khi truy vấn mang tín hiệu từ vịnh

mạnh, lexical giữ ưu thế; khi truy vấn giàu ngữ nghĩa mô tả, dense bù đắp tốt. Sự hỗ trợ này tạo nền tốt hơn cho tầng ranking sau đó.

4.5.3 Slice C: Ranking & Explanation

Thiết kế hard-filter trước scoring giúp loại bỏ mâu thuẫn “điểm cao nhưng không khả thi”. Ở phần explanation, cấu trúc theo thành phần cho phép người dùng nghiệp vụ truy nguyên nhanh lý do xếp hạng và xác định điểm cần hiệu chỉnh dữ liệu. Đây là khác biệt đáng kể so với các danh sách điểm tổng hợp thiếu giải thích.

4.5.4 Slice D: ATS Workflow

State machine giúp các chuyển trạng thái diễn ra nhất quán, giảm lỗi “nhảy bước” và giúp audit dễ hơn. Khi có tranh chấp hoặc cần hậu kiểm, log actor + timestamp + pre/post state cung cấp đường truy vết đầy đủ.

4.5.5 Slice E: Reliability under degradation

Khi mô phỏng lỗi cục bộ ở parse hoặc retrieval, kiến trúc tách lớp cho phép hệ thống suy giảm mềm: một số tác vụ bị chậm hoặc tạm hoãn nhưng không làm toàn bộ pipeline ngừng hoạt động. Đây là thuộc tính quan trọng cho hệ thống tuyển dụng vận hành liên tục.

4.6 Phân tích phản thực (counterfactual) theo baseline

4.6.1 Baseline A: Keyword-only pipeline

Nếu dùng keyword-only làm lõi, hệ thống có thể giảm chi phí suy luận và đơn giản hóa hạ tầng. Tuy nhiên, shortlist dễ lệch về các hồ sơ “khớp chữ” thay vì “khớp nghĩa”,

đặc biệt trong các JD mô tả dài hoặc đa ngữ cảnh. Về dài hạn, bias từ cách viết CV/JD sẽ ảnh hưởng chất lượng matching nhiều hơn mức tiết kiệm chi phí ngắn hạn.

4.6.2 Baseline B: Dense blackbox end-to-end

Nếu dùng dense blackbox end-to-end, chất lượng semantic retrieval có thể cải thiện trong một số kịch bản. Tuy nhiên, hai vấn đề vận hành xuất hiện: khó giải trình cho quyết định nghiệp vụ và khó phân rã lỗi khi đầu vào suy giảm chất lượng. Khi chưa có bộ nhân dày, rủi ro tối ưu sai mục tiêu (offline tốt, online không bền) là đáng kể.

4.6.3 Baseline C: KG-heavy + multi-agent

Nếu đưa KG-heavy và orchestration đa tác tử ngay từ MVP, hệ thống có thể tăng năng lực suy luận tri thức dài hạn. Đổi lại, độ phức tạp triển khai tăng mạnh: ontology design, data governance, đồng bộ tri thức, và kiểm thử liên tác tử. Trong điều kiện MVP, chi phí này khó biện minh so với lợi ích tức thì.

4.6.4 Kết luận phản thực

Các baseline thay thế đều có điểm mạnh cục bộ, nhưng kiến trúc hiện tại đạt cân bằng tốt hơn cho mục tiêu MVP: chất lượng đủ dùng, giải thích được, vận hành ổn định, và có lộ trình nâng cấp từng lớp.

4.7 Threats to Validity

4.7.1 Internal Validity

Rủi ro nội tại lớn nhất nằm ở confounding giữa chất lượng dữ liệu parser và chất lượng ranking. Nếu không tách hai lớp này khi phân tích, kết luận về scorer có thể sai lệch. Cơ chế review gate và logging theo tầng được dùng để giảm rủi ro này, nhưng không loại bỏ hoàn toàn.

4.7.2 Construct Validity

Một số khái niệm như “độ hữu ích của giải thích” khó lượng hóa bằng một chỉ số đơn. Vì vậy, construct này được đo bằng tổ hợp chỉ báo (khả năng truy nguyên quyết định, thời gian phản hồi thao tác, mức nhất quán hành động nghiệp vụ). Dù vậy, vẫn còn rủi ro đo lường chưa bao phủ đủ chiều sâu nhận thức người dùng.

4.7.3 External Validity

Kết quả hiện tại phản ánh dữ liệu và quy trình ở phạm vi MVP. Khi mở rộng sang ngành khác hoặc thị trường ngôn ngữ khác, phân bố dữ liệu và kỳ vọng tuyển dụng có thể thay đổi. Do đó, mọi kết luận tổng quát hóa cần đi kèm tái đánh giá theo miền.

4.7.4 Conclusion Validity

Do thiếu tập nhân quy mô lớn, các kết luận định lượng hiện chưa đạt mức suy diễn thống kê mạnh như các benchmark học thuật chuẩn. Vì vậy, kết luận chương này được đóng khung ở mức “evidence-informed engineering” thay vì “statistically final claims”.

4.8 Bảng tổng hợp mức độ đáp ứng hiện tại

Table 4.2: Tổng hợp mức đáp ứng theo trực đánh giá MVP

Trực đánh giá	Mức	Diễn giải
Độ đúng quy trình	Tốt	Checkpoint trạng thái giúp kiểm soát vòng đời dữ liệu/nghiệp vụ nhất quán.
Độ phủ candidate	Khá-Tốt	Hybrid generation giảm bỏ sót so với một hướng đơn lẻ.
Độ hữu ích shortlist	Khá	Hard-filter + scoring thành phần tạo danh sách gợi ý có thể hành động.
Khả năng giải trình	Khá	Explanation payload hỗ trợ truy nguyên, còn dư địa mở rộng attribution sâu.
Độ bền vận hành	Khá	Tách lớp giúp suy giảm mềm khi lỗi cục bộ, cần tăng telemetry ở tải cao.

Nhận xét tổng quan: hệ thống đạt mục tiêu MVP theo định hướng kỹ thuật thực dụng. Điểm mạnh nằm ở kiến trúc có kiểm soát và khả năng truy vết; điểm cần đầu tư tiếp theo là chuẩn hóa benchmark định lượng có nhãn và tăng chiều sâu đánh giá online.

4.9 Roadmap nghiên cứu và nâng cấp ba giai đoạn

4.9.1 Near-term (0–3 tháng)

- Chuẩn hóa pipeline nhãn top- k theo chuyên gia tuyển dụng.
- Bổ sung dashboard theo tầng: parse quality, retrieval drift, ranking stability.
- Chuẩn hóa bộ câu hỏi phản hồi explanation từ người dùng nghiệp vụ.

4.9.2 Mid-term (3–9 tháng)

- Triển khai benchmark định lượng: precision@ k , recall@ k , nDCG@ k , MRR.

- Chạy ablation cho từng lớp (hard-filter, semantic scoring, rerank signal).
- Thí điểm calibration theo nhóm vị trí để giảm sai lệch liên miền.

4.9.3 Long-term (9+ tháng)

- Thử nghiệm kiểm soát KG augmentation ở domain hẹp.
- Đánh giá khả thi multi-agent orchestration cho các tác vụ phân vai rõ (không thay lỗi ngay).
- Mở rộng đánh giá fairness/ethics theo bối cảnh AI hiring có kiểm chứng dữ liệu thực địa.

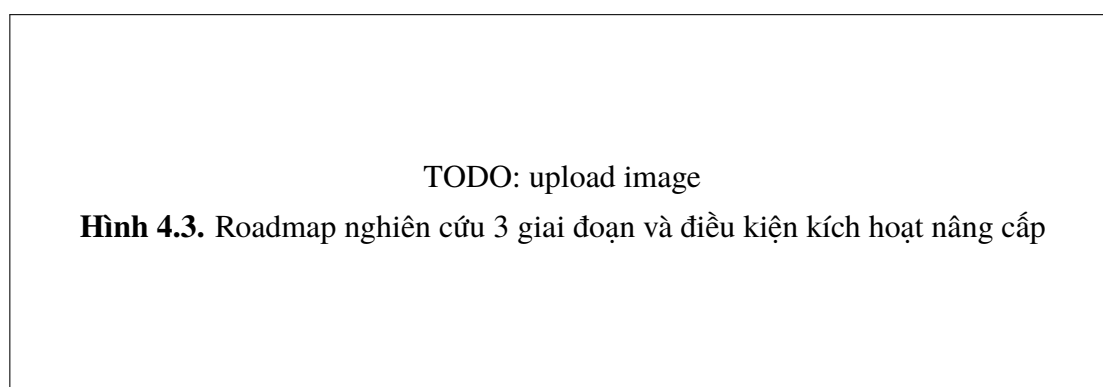


Figure 4.2: Placeholder sơ đồ roadmap nâng cấp theo giai đoạn

4.10 Protocol thực nghiệm chi tiết cho từng RQ

Để tái lập đánh giá, mỗi RQ được gắn với protocol thao tác cụ thể thay vì mô tả tổng quát. Protocol gồm bốn phần: dữ liệu đầu vào, thao tác chạy, điều kiện quan sát, và tiêu chí kết luận.

Table 4.3: Protocol thực nghiệm rút gọn theo RQ

RQ	Input profile	Thao tác chính	Tiêu chí kết luận
RQ1	CV/JD có mức nhiều parser khác nhau	Chạy ingest-parse-review qua nhiều batch	Review gate phải chặn được phần lớn hồ sơ thiếu trường trọng yếu.
RQ2	Bộ truy vấn có và không có hard constraints	Chạy matching với/không có hard-filter	Tỷ lệ cặp vi phạm constraint ở top- k giảm rõ khi bật hard-filter.
RQ3	Tập kết quả có explanation payload đầy đủ	So sánh quyết định thao tác với/không có explanation	Explanation giúp tăng khả năng truy nguyên và nhất quán thao tác.
RQ4	Cùng tập dữ liệu, nhiều baseline kiến trúc	Chạy phân tích phản thực theo giả định kỹ thuật	Cấu hình hiện tại phải chứng minh lợi thế cân bằng, không chỉ một metric đơn lẻ.
RQ5	Kịch bản lỗi cục bộ theo tầng	Inject lỗi có kiểm soát và quan sát phục hồi	Hệ thống phải suy giảm mềm, không sập toàn tuyến.

Protocol chi tiết theo RQ giúp tăng độ tin cậy kết luận và giảm tranh luận cảm tính khi so sánh phương án.

4.11 Phân tích độ nhạy (sensitivity analysis) theo tham số kiến trúc

Một sai lầm thường gặp là xem trọng số scoring hoặc độ sâu rerank như hằng số bất biến. Trên thực tế, các tham số này có độ nhạy cao theo loại vị trí tuyển dụng. Do đó, đánh giá cần xem biến thiên kết quả khi thay đổi có kiểm soát các tham số chính:

- Trọng số lexical/dense/rule adjustment.
- Độ sâu candidate set trước rerank.
- Ngưỡng hard-filter ở các trường điều kiện cứng.
- Chính sách normalization score theo batch.

Table 4.4: Khung sensitivity analysis cho các tham số trọng yếu

Tham số	Khoảng thử	Kỳ vọng quan sát
α lexical weight	0.2 – 0.6	Tăng α giúp truy vấn giàu từ khóa, nhưng có thể giảm bao phủ ngữ nghĩa.
β dense weight	0.3 – 0.7	Tăng β giúp semantic recall, nhưng cần kiểm soát chi phí và độ ổn định explanation.
Rerank depth	20 – 200 ứng viên	Độ sâu lớn có thể tăng chất lượng top- k nhưng tăng độ trễ đáng kể.
Constraint threshold	mềm – chặt	Ngưỡng chặt giảm false positive nhưng có thể tăng false rejection ở trường hợp biên.

Phân tích độ nhạy không nhằm tìm một bộ tham số “đúng tuyệt đối”, mà nhằm xác định vùng tham số an toàn cho vận hành.

4.12 Ablation framework cho kiến trúc nhiều tầng

Ablation là công cụ quan trọng để định lượng đóng góp của từng lớp trong pipeline. Với JobConnect, ablation nên chạy theo thứ tự tăng dần độ phức tạp:

- Cấu hình A0: lexical + hard-filter.
- Cấu hình A1: A0 + dense retrieval.
- Cấu hình A2: A1 + hybrid merge policy.
- Cấu hình A3: A2 + rerank signal.
- Cấu hình A4: A3 + explanation-by-construction đầy đủ.

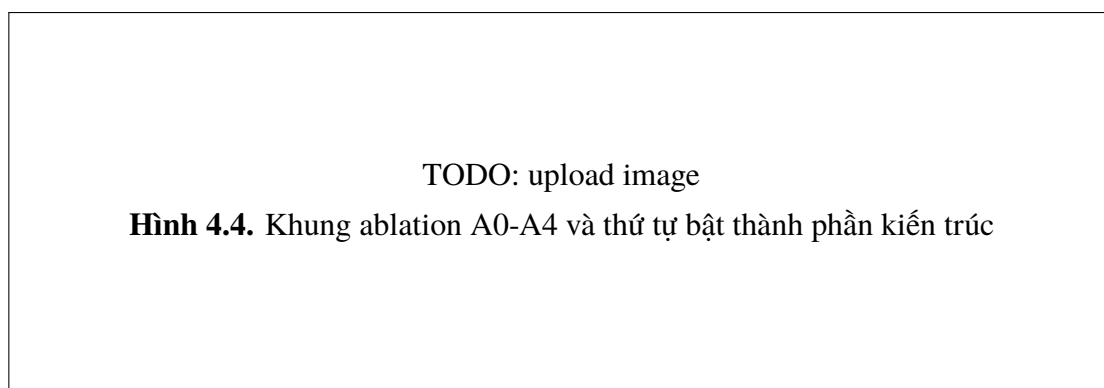


Figure 4.3: Placeholder sơ đồ ablation framework cho pipeline nhiều tầng

Khung ablation này giúp trả lời câu hỏi thực dụng: thành phần nào tạo giá trị thực, thành phần nào chỉ tăng độ phức tạp mà lợi ích biên thấp.

4.13 Khuyến nghị chuẩn hóa báo cáo kết quả cho hội đồng

Để báo cáo có tính học thuật cao và dễ phản biện, phần kết quả nên chuẩn hóa theo mẫu:

- Nêu rõ giả thiết kỹ thuật trước mỗi bảng/biểu đồ.
- Tách riêng kết quả quan sát và diễn giải nguyên nhân.
- Với mỗi kết luận, chỉ ra metric và nguồn evidence tương ứng.
- Tránh tuyên bố tổng quát vượt quá phạm vi dữ liệu đánh giá.

Chuẩn hóa mẫu báo cáo giúp giữ nhất quán giữa các vòng cập nhật luận văn, đồng thời giảm rủi ro lập luận vòng tròn hoặc suy diễn quá mức từ dữ liệu hạn chế.

4.14 Rủi ro triển khai khi mở rộng từ MVP lên production

Khi chuyển từ MVP sang production, ba rủi ro kỹ thuật xuất hiện sớm:

- **Scale risk:** độ trễ tăng do số lượng hồ sơ và truy vấn tăng cùng lúc.
- **Governance risk:** thay đổi policy tuyển dụng nhanh hơn vòng cập nhật scoring rule.
- **Model lifecycle risk:** phiên bản mô hình/embedding không được quản trị nhất quán.

Để giảm rủi ro, đề xuất tối thiểu là: version hóa policy và model, tách môi trường benchmark khỏi môi trường vận hành, và thêm cơ chế canary cho thay đổi scoring logic.

4.15 Governance thực nghiệm và khả năng tái lập kết quả

Một báo cáo học thuật mạnh cần khả năng tái lập. Với hệ thống ứng dụng như JobConnect, tái lập không chỉ là chạy lại code mà còn là tái lập đúng ngữ cảnh dữ liệu, phiên bản policy và cấu hình pipeline.

Table 4.5: Experiment card tối thiểu cho mỗi vòng đánh giá

Trường bắt buộc	Nội dung cần ghi nhận
Dataset slice	Nguồn CV/JD, khoảng thời gian snapshot, tỷ lệ hồ sơ thiếu trường, mức nhiễu parser.
Version bundle	Policy version, model version, index version, cấu hình hard-filter/rerank.
RQ target	Câu hỏi nghiên cứu chính của lượt chạy và giả thuyết cần kiểm chứng.
Metrics	Bộ metric chính/phụ, ngưỡng chấp nhận, phương pháp tổng hợp.
Outcome notes	Kết quả định lượng, quan sát định tính, quyết định giữ/đổi cấu hình.
Risk notes	Các bất thường, giới hạn dữ liệu, mức độ tin cậy kết luận.

Chuẩn hóa experiment card giúp hội đồng hoặc nhóm vận hành kiểm tra được đường suy luận từ dữ liệu đến kết luận, giảm rủi ro “kết luận đúng do may mắn dữ liệu”.

4.16 Phân tích chi phí vận hành theo baseline và ngưỡng đầu tư

Ngoài metric chất lượng, luận văn cần làm rõ câu hỏi: khi nào nên đầu tư sang cấu hình phức tạp hơn? Ta định nghĩa chi phí kỳ vọng theo truy vấn:

$$E[\text{Cost/query}] = c_r + d \cdot c_{rr} + c_o \quad (4.1)$$

trong đó c_r là chi phí retrieval nền, d là rerank depth, c_{rr} là chi phí rerank trên mỗi ứng viên, và c_o là overhead observability/governance.

Lợi ích biên của cấu hình nâng cấp được xem xét qua:

$$\Delta U = \Delta Q - \gamma \cdot \Delta \text{Cost} \quad (4.2)$$

Nếu $\Delta U > 0$ ổn định trên nhiều batch và nhiều RQ, nâng cấp có thể được chấp nhận. Nếu không, cấu hình hiện tại vẫn là lựa chọn tối ưu trong ngữ cảnh MVP.

Table 4.6: Khung quyết định đầu tư theo lợi ích biên

Tình huống	Dấu hiệu định lượng	Khuyến nghị
Nâng cấp rerank sâu	ΔQ nhỏ, ΔCost lớn	Giữ cấu hình hiện tại, chỉ tăng depth cục bộ theo nhóm truy vấn khó.
Thêm attribution phức tạp	Explainability tăng nhẹ, latency tăng đáng kể	Hoãn triển khai rộng, thử nghiệm trên domain hẹp có nhu cầu audit cao.
Tối ưu hybrid fusion	ΔQ và stability cùng tăng, cost tăng thấp	Ưu tiên triển khai, vì cải thiện utility tổng thể.

4.17 Phân tích lỗi theo error bucket và giá trị cho roadmap

Phân tích lỗi nên đi theo “error bucket” thay vì chỉ liệt kê case lẻ:

- **Bucket E1 – Data quality:** thiếu trường kỹ năng, kinh nghiệm, hoặc metadata ngữ cảnh.
- **Bucket E2 – Retrieval miss:** candidate set bỏ sót ứng viên phù hợp do lexical/dense lệch pha.
- **Bucket E3 – Ranking disorder:** candidate đúng có mặt nhưng thứ hạng không ổn định ở top- k .
- **Bucket E4 – Explanation inconsistency:** điểm thành phần và narrative không đồng nhất.
- **Bucket E5 – Workflow friction:** kết quả đúng kỹ thuật nhưng khó dùng trong thao tác ATS.

Mỗi bucket cần ánh xạ trực tiếp vào roadmap:

- E1 ưu tiên near-term vì tác động diện rộng.
- E2/E3 ưu tiên mid-term với ablation và sensitivity có kiểm soát.
- E4/E5 là nền cho long-term governance và trust.

TODO: upload image

Hình 4.5. Bản đồ error bucket và ánh xạ sang roadmap 3 giai đoạn

Figure 4.4: Placeholder bản đồ lỗi hệ thống và ưu tiên xử lý theo giai đoạn

Khung error bucket giúp biến phần “hạn chế” từ mô tả thụ động thành chương trình cải tiến chủ động có thứ tự ưu tiên rõ ràng.

4.18 Khung báo cáo negative results và quyết định dừng sớm

Trong nghiên cứu ứng dụng, không phải mọi hướng nâng cấp đều tạo cải thiện đáng kể. Việc ghi nhận “negative results” (kết quả không cải thiện) có vai trò quan trọng vì giúp tránh lặp lại thử nghiệm tốn kém và tăng chất lượng quyết định đầu tư.

Đối với JobConnect, một vòng thử nghiệm nên được đánh dấu là negative result khi thỏa đồng thời:

- Cải thiện chất lượng không vượt ngưỡng tối thiểu đã định trước.
- Độ trễ hoặc chi phí tăng vượt ngưỡng vận hành chấp nhận.
- Explainability không tốt hơn hoặc khó truy nguyên hơn cấu hình hiện tại.

Khi xuất hiện negative result, hành động phù hợp không phải luôn là “thử lại ngay”, mà là phân loại nguyên nhân:

- **Do dữ liệu:** cần cải thiện parse/normalize trước khi đánh giá lại mô hình.
- **Do cấu hình:** giữ mô hình nhưng đổi độ sâu rerank hoặc trọng số fusion.
- **Do giả thuyết sai:** đóng nhánh thử nghiệm và chuyển tài nguyên sang hướng khác.

Table 4.7: Quy tắc dừng sớm cho thử nghiệm nâng cấp

Tín hiệu	Mức độ	Quyết định
ΔQ thấp liên tiếp nhiều batch	Cao	Dừng thử nghiệm, quay về baseline ổn định, lưu biên bản negative result.
Chi phí tăng nhanh hơn lợi ích	Trung bình-Cao	Giảm phạm vi thử nghiệm hoặc hạ cấu hình để xác định ngưỡng hiệu quả.
Kết quả dao động mạnh giữa batch	Trung bình	Tăng cỡ mẫu và chuẩn hóa protocol trước khi kết luận.

Khung dừng sớm giúp Chương 4 không chỉ trả lời “hướng nào tốt” mà còn trả lời “hướng nào nên dừng”, từ đó tăng tính thực dụng và độ tin cậy của roadmap nghiên cứu.

4.19 Kết luận chương

Chương 4 đã chuyển đánh giá từ mức “hệ thống chạy được” sang mức “hệ thống hợp lý về mặt kiến trúc và vận hành dưới ràng buộc MVP”. Phân tích phản thực cho thấy cấu hình hiện tại không cực đại thuật toán, nhưng tối ưu hơn về cân bằng chất lượng–chi phí–giải trình. Phần threats to validity và roadmap ba giai đoạn cung cấp nền tảng rõ ràng cho các vòng nghiên cứu và nâng cấp tiếp theo.

Kết luận

Kết quả đạt được

Khóa luận đã xây dựng được một khung xử lý tuyển dụng số theo hướng cân bằng giữa tính chính xác kỹ thuật và khả năng giải thích nghiệp vụ. Kết quả quan trọng nhất là mô hình hai lớp đánh giá: lớp điều kiện bắt buộc để đảm bảo tính khả thi và lớp tương đồng ngữ nghĩa để xếp hạng mức độ phù hợp. Cách tổ chức này giúp kết quả so khớp có căn cứ rõ ràng, tránh phụ thuộc hoàn toàn vào suy luận khó kiểm soát.

Bên cạnh đó, nghiên cứu đã chuẩn hóa được cách nhìn hệ thống ở mức vận hành: dữ liệu đầu vào được xử lý theo tiến trình trạng thái, tương tác tuyển dụng được theo dõi theo vòng đời, và các sự kiện quan trọng có thể truy vết để phục vụ kiểm tra hoặc cải tiến. Đây là nền tảng cần thiết để chuyển từ bản thử nghiệm sang sản phẩm có khả năng vận hành ổn định.

Hạn chế

Hạn chế lớn nhất của nghiên cứu nằm ở dữ liệu đánh giá. Khi chưa có tập dữ liệu gán nhãn đủ rộng, việc kết luận chất lượng xếp hạng ở mức tổng quát còn thận trọng. Ngoài ra, dù cơ chế giải thích đã hình thành, cách diễn giải cho người dùng không chuyên vẫn cần được chuẩn hóa sâu hơn để tăng tính nhất quán trong trải nghiệm thực tế.

Một hạn chế khác là độ bao phủ của các tình huống biên trong vận hành. Những trường hợp dữ liệu thiếu, dữ liệu nhiễu cao hoặc biến thể ngôn ngữ đặc thù vẫn cần được

quan sát thêm qua các vòng thử nghiệm mở rộng.

Hướng phát triển

Hướng phát triển thứ nhất là xây dựng bộ dữ liệu gắn nhãn theo chuyên gia tuyển dụng để đánh giá định lượng có cơ sở hơn. Hướng thứ hai là thực hiện thí nghiệm ablation nhằm đo tác động riêng của từng thành phần trong mô hình, từ đó hiệu chỉnh trọng số theo bằng chứng thay vì kinh nghiệm chủ quan.

Hướng thứ ba là mở rộng cơ chế quan sát vận hành theo thời gian thực để phát hiện sớm sai lệch dữ liệu và suy giảm chất lượng xử lý. Hướng thứ tư là chuẩn hóa ngôn ngữ giải thích cho từng nhóm người dùng, bảo đảm kết quả vừa đúng về kỹ thuật vừa dễ hiểu trong ngữ cảnh nghiệp vụ.

Kết luận chung

Tổng thể, khóa luận đã chứng minh tính khả thi của cách tiếp cận tuyển dụng số dựa trên mô hình xử lý có giải thích. Đóng góp của đề tài không nằm ở một thuật toán đơn lẻ, mà ở cách thiết kế hệ thống theo chuỗi xử lý minh bạch, có kiểm soát trạng thái và có khả năng truy vết. Với các hướng phát triển đã nêu, mô hình này có thể tiếp tục mở rộng để đáp ứng yêu cầu thực tế ở quy mô lớn hơn.

Bibliography

- [1] JobConnect repository, “REQUIREMENTS.md – Production MVP Recruiting Marketplace”, tài liệu yêu cầu sản phẩm nội bộ.
- [2] JobConnect repository, “Production HLD: Overview And Problem”, tài liệu kiến trúc mức cao.
- [3] JobConnect repository, “Production HLD: Architecture Overview”, tài liệu kiến trúc backend.
- [4] JobConnect repository, “Production HLD: Data And Storage”, tài liệu thiết kế dữ liệu và lưu trữ.
- [5] JobConnect repository, “Production HLD: Matching And Search Pipeline”, tài liệu thiết kế matching và search.
- [6] JobConnect repository, “Production HLD: API And Runtime Flows”, tài liệu luồng runtime và API.
- [7] JobConnect repository, “Production HLD: Security, Notifications, And Audit”, tài liệu bảo mật, notification và audit.
- [8] JobConnect repository, “Production LLD: API Contract”, tài liệu hợp đồng API production.
- [9] JobConnect repository, “Current API Implementation Matrix”, tài liệu đối chiếu hiện trạng triển khai với target contract.
- [10] Greenhouse, “Greenhouse platform”, <https://www.greenhouse.com/platform>.

- [11] Lever, “LeverTRM ATS + CRM”, <https://www.lever.co/lever-trm>.
- [12] Workday, “Workday Talent Acquisition”, <https://www.workday.com/en-us/products/talent-management/talent-acquisition.html>.
- [13] Stephen Robertson and Hugo Zaragoza, “The Probabilistic Relevance Framework: BM25 and Beyond”, *Foundations and Trends in Information Retrieval*, 3(4):333–389, 2009. DOI: <https://doi.org/10.1561/15000000019>.
- [14] Vladimir Karpukhin, Barlas Öğuz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih, “Dense Passage Retrieval for Open-Domain Question Answering”, arXiv:2004.04906, 2020. <https://arxiv.org/abs/2004.04906>.
- [15] Nils Reimers and Iryna Gurevych, “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks”, arXiv:1908.10084, 2019. <https://arxiv.org/abs/1908.10084>.
- [16] Dor Lavi, Volodymyr Medentsiy, and David Graus, “conSultantBERT: Fine-tuned Siamese Sentence-BERT for Matching Jobs and Job Seekers”, arXiv:2109.06501, 2021. <https://arxiv.org/abs/2109.06501>.
- [17] Xiang Li, Niyu Ge, Chunyan Xiao, and Yen-Yu Chen, “Matching Resumes to Jobs via Deep Siamese Network”, *Companion Proceedings of The Web Conference (WWW)*, 2018. <https://research.ibm.com/publications/matching-resumes-to-jobs-via-deep-siamese-network>.
- [18] Thibault Formal, Benjamin Piwowarski, and Stéphane Clinchant, “SPLADE: Sparse Lexical and Expansion Model for First Stage Ranking”, arXiv:2107.05720, 2021. <https://arxiv.org/abs/2107.05720>.
- [19] Keshav Santhanam, Omar Khattab, Jon Saad-Falcon, Christopher Potts, and Matei Zaharia, “ColBERTv2: Effective and Efficient Retrieval via Lightweight Late Interaction”, *NAACL-HLT*, 2022. DOI: <https://doi.org/10.18653/v1/2022.naacl-main.272>.
- [20] Jing Lu, Keith Hall, Ji Ma, and Jianmo Ni, “HYRR: Hybrid Infused Reranking for Passage Retrieval”, arXiv:2212.10528, 2022. <https://arxiv.org/abs/2212.10528>.

- [21] Qiang Wu, Christopher J. C. Burges, Krysta M. Svore, and Jianfeng Gao, “Adapting Boosting for Information Retrieval Measures” (LambdaMART), *Information Retrieval*, 13:254–270, 2010. DOI: <https://doi.org/10.1007/s10791-009-9112-1>.
- [22] Tanya Chowdhury, Razieh Rahimi, and James Allan, “Rank-LIME: Local Model-Agnostic Feature Attribution for Learning to Rank”, *arXiv:2212.12722*, 2022. <https://arxiv.org/abs/2212.12722>.
- [23] Sara Kassir, Lewis Baker, Jackson Dolphin, and Frida Polli, “AI for hiring in context: a perspective on overcoming the unique challenges of employment research to mitigate disparate impact”, *AI and Ethics*, 3:845–868, 2023. DOI: <https://doi.org/10.1007/s43681-022-00208-x>.
- [24] Lisa T. T. Köchling and Marius Wehner, “Ethics of AI-Enabled Recruiting and Selection: A Review and Research Agenda”, *Journal of Business Ethics*, 178:977–1007, 2022. DOI: <https://doi.org/10.1007/s10551-022-05049-6>.

APPENDIX A

Phụ lục mô hình xử lý và truy vết nghiên cứu

A.1 Bản đồ chức năng ở mức khái niệm

Phụ lục này trình bày bản đồ chức năng ở mức khái niệm nhằm hỗ trợ đối chiếu nhanh khi đọc báo cáo. Nội dung tập trung vào vai trò nghiệp vụ, dữ liệu vào/ra và nguyên tắc vận hành, không mô tả chi tiết triển khai kỹ thuật.

Table A.1: Nhóm chức năng và mục tiêu nghiệp vụ

Nhóm chức năng	Mục tiêu nghiệp vụ
Quản lý tài khoản và vai trò	Bảo đảm mỗi thao tác có chủ thể hợp lệ và quyền truy cập phù hợp.
Quản lý hồ sơ ứng viên	Chuẩn hóa và duy trì thông tin ứng viên theo vòng đời trạng thái.
Quản lý nhu cầu tuyển dụng	Công bố và duy trì thông tin vị trí tuyển dụng theo vòng đời trạng thái.
Chuẩn hóa dữ liệu tài liệu	Chuyển đổi dữ liệu phi cấu trúc thành biểu diễn có cấu trúc để so khớp.
So khớp có giải thích	Tạo danh sách gợi ý có thứ tự và nêu rõ căn cứ đánh giá.
Tương tác tuyển dụng	Quản lý ứng tuyển, lời mời và cập nhật trạng thái nghiệp vụ.
Quan sát vận hành	Ghi nhận sự kiện, cảnh báo và hỗ trợ truy vết khi phát sinh sự cố.

A.2 Mô hình dữ liệu ở mức nghiệp vụ

Table A.2: Nhóm dữ liệu và ràng buộc cốt lõi

Nhóm dữ liệu	Ràng buộc cốt lõi
Định danh và quyền	Mỗi bản ghi phải gắn vai trò và trạng thái truy cập rõ ràng.
Hồ sơ ứng viên và vị trí tuyển dụng	Chỉ bản ghi ở trạng thái sẵn sàng mới tham gia không gian so khớp công khai.
Dữ liệu xử lý tài liệu	Mỗi lần xử lý cần có trạng thái tiến trình và thông tin lỗi khi thất bại.
Dữ liệu so khớp	Kết quả phải lưu được điểm tổng, điểm thành phần và thông tin giải thích tối thiểu.
Dữ liệu tương tác tuyển dụng	Mỗi thay đổi trạng thái phải có dấu vết theo thời gian để phục vụ truy vết.

A.3 Khung giải thích kết quả so khớp

Table A.3: Thành phần giải thích bắt buộc

Thành phần	Ý nghĩa
Kết quả điều kiện bắt buộc	Xác định cặp dữ liệu có đủ điều kiện tham gia xếp hạng hay không.
Điểm thành phần	Cho biết mức đóng góp tương đối của từng góc nhìn đánh giá.
Điểm tổng hợp	Thể hiện mức phù hợp cuối cùng sau khi tổng hợp các thành phần.
Nhận xét giải thích	Tóm tắt điểm mạnh, điểm hạn chế và nguyên nhân chính của thứ hạng.
Cảnh báo dữ liệu	Chỉ ra thông tin thiếu hoặc nhiễu có thể làm giảm độ tin cậy kết quả.

A.4 Khung truy vết nguồn tri thức

Table A.4: Nguồn tri thức và mục đích sử dụng trong báo cáo

Nguồn tri thức	Mục đích sử dụng
Đặc tả yêu cầu sản phẩm	Khóa mục tiêu, phạm vi nghiên cứu và tiêu chí chấp nhận.
Tài liệu kiến trúc tổng thể	Định hình thành phần xử lý, luồng dữ liệu và nguyên tắc thiết kế.
Tài liệu hợp đồng dữ liệu	Chuẩn hóa cách hiểu dữ liệu vào/ra và quy tắc xử lý lỗi.
Tài liệu hiện trạng triển khai	Đối chiếu mức độ hoàn thành và nhận diện khoảng trống cải tiến.
Bộ quy tắc viết báo cáo	Bảo đảm cấu trúc học thuật, giọng văn và tính nhất quán trình bày.

A.5 Checklist duy trì chất lượng báo cáo

Table A.5: Checklist kiểm soát chất lượng khi cập nhật báo cáo

STT	Tiêu chí kiểm tra
1	Đề mục thống nhất màu sắc, cấp độ và ngôn ngữ trình bày.
2	Toàn văn dùng một font chữ và không lạm dụng định dạng đặc biệt cho thuật ngữ.
3	Nội dung tập trung vào mô hình xử lý bài toán, không sa vào mô tả chi tiết triển khai.
4	Bảng biểu không tràn khổ trang và có cấu trúc dễ đọc.
5	Mọi bảng kết quả chính đều có đoạn nhận xét xu hướng và nguyên nhân.
6	Trích dẫn tài liệu được sử dụng nhất quán theo định dạng đã chọn.
7	Hạn chế và hướng phát triển được nêu rõ, có liên hệ với kết quả đánh giá.